

Vel Tech Group of Institutions

Name: KARTHIK YARNAGULA

Email: vtu30340@veltech.edu.in

Roll no: vtu30340

Phone: null

Branch: VTU

Department: CSE with CS

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 12_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Meera is preparing for a science exhibition and needs to select a group of items such that their total weight is exactly equal to a given target weight. Each item has a known weight, and every item can be selected at most once.

Given a list of item weights and a target weight k , determine whether there exists a subset of items whose total weight is exactly k .

If such a subset exists, print any one valid subset.

If no such subset exists, print -1.

The solution must be implemented using Dynamic Programming.

Formula

Let

$dp[i][j] = \text{true}$

if a subset of the first i items can form a sum j .

Transition:

If $arr[i-1] > j$

$dp[i][j] = dp[i-1][j]$

Else

$dp[i][j] = dp[i-1][j] \text{ OR } dp[i-1][j - arr[i-1]]$

Input Format

The first line contains two integers n and k , where n is the number of items and k is the target weight.

The second line contains n space-separated integers representing item weights.

Output Format

If a valid subset exists, print the elements of one such subset in a single line separated by spaces.

If no subset exists, print -1.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5 9
3 34 4 12 5
Output: 5 4

Answer

// You are using GCC
#include <stdio.h>

```
int main() {
    int n, k;
    scanf("%d %d", &n, &k);

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // DP table
    int dp[n + 1][k + 1];

    // Initialize
    for (int i = 0; i <= n; i++) {
        dp[i][0] = 1; // sum 0 is always possible
    }
    for (int j = 1; j <= k; j++) {
        dp[0][j] = 0;
    }

    // Fill DP table
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= k; j++) {
            if (arr[i - 1] > j) {
                dp[i][j] = dp[i - 1][j];
            } else {
                dp[i][j] = dp[i - 1][j] || dp[i - 1][j - arr[i - 1]];
            }
        }
    }

    // If no subset exists
    if (dp[n][k] == 0) {
        printf("-1");
    }
}
```

```

    return 0;
}

// Backtrack to find subset
int i = n, j = k;

while (i > 0 && j > 0) {
    // If value comes from top element not included
    if (dp[i - 1][j]) {
        i--;
    } else {
        // Element included
        printf("%d ", arr[i - 1]);
        j -= arr[i - 1];
        i--;
    }
}

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Ramya is working as a logistics coordinator for a delivery company. Every day, she receives a list of delivery locations arranged along a straight highway. Each location has a certain delivery priority value (which may be positive or negative depending on profit or loss).

Ramya wants to choose a continuous sequence of delivery locations such that the total delivery priority is maximized, ensuring maximum profit for that day.

Since the company handles a large number of deliveries, Ramya needs an efficient solution that runs in polynomial time.

Input Format

The first line contains an integer n , representing the number of delivery locations.

The second line contains n space-separated integers, representing the priority values.

Output Format

The output prints a single integer representing the maximum total priority of any contiguous sequence of locations.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 8

-2 3 5 -1 4 -5 2 6

Output: 14

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int arr[n];
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    int current_sum = arr[0];
```

```
    int max_sum = arr[0];
```

```
    for (int i = 1; i < n; i++) {
```

```
        // Choose max of current element or extending previous sum
```

```
        if (current_sum + arr[i] > arr[i]) {
```

```
            current_sum = current_sum + arr[i];
```

```
        } else {
```

```
            current_sum = arr[i];
```

```
        }
```

```
// Update global maximum
if (current_sum > max_sum) {
    max_sum = current_sum;
}
}

printf("%d", max_sum);

return 0;
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Ragav is managing a chain of warehouses located along a long highway. Each warehouse is positioned at a specific coordinate on the road.

To improve efficiency, Ragav wants to choose a central hub location such that the total travel distance from all warehouses to this hub is minimized.

He can place the hub at any one of the existing warehouse locations.

Since the number of warehouses can be very large, Ragav needs an efficient algorithm that runs in polynomial time to compute the optimal location.

Input Format

The first line contains an integer n , representing the number of warehouses.

The second line contains n space-separated integers, representing the positions of warehouses on the highway.

Output Format

The output prints a single integer representing the minimum total distance from all warehouses to the chosen hub.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

1 2 3 6 10

Output: 13

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Comparator for qsort
```

```
int compare(const void *a, const void *b) {  
    return (*(int*)a - *(int*)b);  
}
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int arr[n];
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    // Sort the array
```

```
    qsort(arr, n, sizeof(int), compare);
```

```
    // Find median
```

```
    int median = arr[n / 2];
```

```
    long long totalDistance = 0;
```

```
    // Calculate total distance
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (arr[i] > median)
```

```
            totalDistance += (arr[i] - median);
```

```
        else
```

```
            totalDistance += (median - arr[i]);
```

```
    }
```

```
printf("%lld", totalDistance);  
    return 0;  
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Peter is a project manager in a software company. He has a list of employees, and each employee has a certain skill contribution value (which can be positive or negative depending on their impact on the project).

Peter wants to form a team such that the total skill contribution is exactly equal to a given target value. Each employee can be selected at most once, and the team must contain at least one employee.

Since the number of possible teams grows rapidly with the number of employees, Peter needs to determine whether such a team exists.

Input Format

The first line contains an integer n , representing the number of employees.

The second line contains n space-separated integers, representing the skill contribution of each employee.

The third line contains an integer $target$, representing the required total skill.

Output Format

If a valid team exists, the output prints: "Team Found: x_1 x_2 x_3 ..."

If no such team exists, the output prints: "No valid team found"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5

3 7 -2 5 1

6

Output: Team Found: 3 -2 5

Answer

// You are using GCC

#include <stdio.h>

int found = 0;

void findTeam(int arr[], int n, int index, int target, int subset[], int size) {

// If target achieved and at least one member

if (target == 0 && size > 0) {

printf("Team Found: ");

for (int i = 0; i < size; i++) {

printf("%d ", subset[i]);

}

printf("\n");

found = 1;

return;

}

// Stop if end reached

if (index == n) return;

// Include current employee

subset[size] = arr[index];

findTeam(arr, n, index + 1, target - arr[index], subset, size + 1);

if (found) return; // stop further search

// Exclude current employee

findTeam(arr, n, index + 1, target, subset, size);

}

int main() {

int n;

scanf("%d", &n);

```

int arr[n];
for (int i = 0; i < n; i++) {
    scanf("%d", &arr[i]);
}

int target;
scanf("%d", &target);

int subset[n];

findTeam(arr, n, 0, target, subset, 0);

if (!found) {
    printf("No valid team found");
}

return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

Viyana is an event coordinator organizing multiple sessions for a tech conference. Each session has a start time and an end time.

Due to limited resources, Viyana wants to select a group of sessions such that:

No two selected sessions overlap in time. The total number of selected sessions is maximized.

However, to make the event more impactful, each session also has a priority score, and Viyana wants to maximize the total priority score instead of just the count.

Your task is to help Viyana select the optimal set of non-overlapping sessions that yields the maximum total priority score.

Input Format

The first line contains an integer n, representing the number of sessions.

The next n lines each contain three integers: start time, end time and priority score.

Output Format

The first line of output prints the maximum total priority score.

In the next line, print the selected sessions (indices starting from 1) in ascending order.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

1 3 5

2 5 6

4 6 5

6 7 4

Output: 14

1 3 4

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
int n;
```

```
int start[25], end[25], priority[25];
```

```
int maxProfit = 0;
```

```
int bestSet[25], bestSize = 0;
```

```
// Check if current session is compatible with selected ones
```

```
int isValid(int selected[], int size, int idx) {
```

```
    for (int i = 0; i < size; i++) {
```

```
        int j = selected[i];
```

```

// overlap condition
if (!(end[j] <= start[idx] || start[j] >= end[idx])) {
    return 0;
}
}
return 1;
}

void solve(int index, int selected[], int size, int currentProfit) {

    // Update best result
    if (currentProfit > maxProfit) {
        maxProfit = currentProfit;
        bestSize = size;
        for (int i = 0; i < size; i++) {
            bestSet[i] = selected[i];
        }
    }

    if (index == n) return;

    // Include current session if valid
    if (isValid(selected, size, index)) {
        selected[size] = index;
        solve(index + 1, selected, size + 1, currentProfit + priority[index]);
    }

    // Exclude current session
    solve(index + 1, selected, size, currentProfit);
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d %d %d", &start[i], &end[i], &priority[i]);
    }

    int selected[25];

    solve(0, selected, 0, 0);
}

```

```
// Output result
printf("%d\n", maxProfit);

// Print indices (1-based)
for (int i = 0; i < bestSize; i++) {
    printf("%d ", bestSet[i] + 1);
}

return 0;
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KARTHIK YARNAGULA

Email: vtu30340@veltech.edu.in

Roll no: vtu30340

Phone: null

Branch: VTU

Department: CSE with CS

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 12_COD

Attempt : 1

Total Mark : 30

Marks Obtained : 29

Section 1 : Coding

1. Problem Statement

Raj is developing financial software that must identify all non-empty subsets of a given list of expenses whose total sum equals a specified target value.

Each expense can either be included or excluded from a subset. The same expense cannot be used more than once in a subset.

Your task is to print all such valid subsets. If no valid non-empty subset exists whose sum equals the target value, print:

No subset matches the target sum

This problem demonstrates how the number of possible subsets increases exponentially when using brute-force approaches.

Formula

Subset Sum = sum of selected elements in the subset

A subset is considered valid if:

$\text{sum}(\text{subset elements}) = \text{target}$

Input Format

The first line contains an integer n , representing the number of expenses.

The second line contains n space-separated integers representing the expense values.

The third line contains an integer t , representing the target sum.

Output Format

Print each valid non-empty subset whose elements sum to t , one subset per line.

Elements of a subset must be printed as space-separated integers.

If no such subset exists, print:

No subset matches the target sum

Refer to the sample output for formatting.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

1 2 3

3

Output: 1 2

3

Answer

```
// You are using GCC
#include <stdio.h>
```

```
int found = 0;
```

```
void findSubsets(int arr[], int n, int index, int target, int subset[], int size) {
```

```
    // If target becomes 0 print subset
```

```
    if (target == 0 && size > 0) {
```

```
        found = 1;
```

```
        for (int i = 0; i < size; i++) {
```

```
            printf("%d ", subset[i]);
```

```
        }
```

```
        printf("\n");
```

```
        return; // STOP further exploration from this path
```

```
    }
```

```
    // Stop conditions
```

```
    if (index == n) {
```

```
        return;
```

```
    }
```

```
    // Include current element
```

```
    subset[size] = arr[index];
```

```
    findSubsets(arr, n, index + 1, target - arr[index], subset, size + 1);
```

```
    // Exclude current element
```

```
    findSubsets(arr, n, index + 1, target, subset, size);
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int arr[n];
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    int target;
```

```
    scanf("%d", &target);
```



```
int subset[n];

findSubsets(arr, n, 0, target, subset, 0);

if (!found) {
    printf("No subset matches the target sum");
}

return 0;
}
```

Status : Partially correct

Marks : 9/10

2. Problem Statement

Sanjana is a cybersecurity analyst responsible for securing communication in a corporate network. The network consists of multiple computers (nodes) connected by communication links (edges).

To ensure secure communication, Sanjana wants to select a group of computers such that every pair of selected computers is directly connected to each other. This ensures a fully trusted group where all members can communicate securely without intermediaries. Such a group is called a clique.

Sanjana is given a list of connections in the network, and she needs to determine whether there exists a clique of size k .

Input Format

The first line contains two integers n and m , representing the number of computers (nodes) and connections (edges).

The next m lines each contain two integers u and v , representing a connection between nodes u and v .

The last line contains an integer k , representing the required clique size.

Output Format

If a clique of size k exists, the output prints: "Clique Found: v1 v2 v3 ..."

Otherwise, the output prints: "No clique of given size exists"

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5 6

0 1

0 2

1 2

1 3

2 3

3 4

3

Output: Clique Found: 0 1 2

Answer

// You are using GCC

#include <stdio.h>

int graph[20][20];

int clique[20];

int n, k;

int found = 0;

// Check if current node can be added to clique

int isSafe(int v, int size) {

for (int i = 0; i < size; i++) {

if (graph[v][clique[i]] == 0) {

return 0;

}

}

return 1;

}

// Backtracking function

void findClique(int start, int size) {

```

        // If clique of size k found
        if (size == k) {
            found = 1;
            printf("Clique Found: ");
            for (int i = 0; i < k; i++) {
                printf("%d ", clique[i]);
            }
            printf("\n");
            return;
        }

        for (int i = start; i < n; i++) {
            if (isSafe(i, size)) {
                clique[size] = i;
                findClique(i + 1, size + 1);

                if (found) return; // stop after first clique
            }
        }
    }
}

```

```

int main() {
    int m, u, v;

    scanf("%d %d", &n, &m);

    // Initialize graph
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            graph[i][j] = 0;
        }
    }

    // Input edges
    for (int i = 0; i < m; i++) {
        scanf("%d %d", &u, &v);
        graph[u][v] = 1;
        graph[v][u] = 1; // undirected graph
    }

    scanf("%d", &k);
}

```

```
findClique(0, 0);  
if (!found) {  
    printf("No clique of given size exists");  
}  
  
return 0;  
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Rehan, a logistics manager, is packing valuable items into two separate containers. Each item has a specific value, and he wants both containers to carry the same total value to ensure balance and fairness. He needs to decide if it's possible to divide the items this way and print one such valid division if it exists.

This problem is based on a classic NP-complete problem known as the Partition Problem, where finding a solution may require checking all combinations in the worst case.

Help him write a program to solve this problem.

Input Format

The first line of input consists of an integer n , representing the number of items.

The second line contains n space-separated integers, where each integer represents the value of an item.

Output Format

If a valid equal partition exists, print:

- Subset 1: $x_1 x_2 x_3 \dots$
- Subset 2: $y_1 y_2 y_3 \dots$

If no such partition exists, it prints "No valid equal partition exists."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

1 5 11 5

Output: Subset 1: 1 5 5

Subset 2: 11

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
int found = 0;
```

```
// Function to find subset with given target
```

```
void findSubset(int arr[], int n, int index, int target, int subset[], int size, int used[]) {
```

```
    if (target == 0) {  
        found = 1;
```

```
        // Print Subset 1
```

```
        printf("Subset 1: ");
```

```
        for (int i = 0; i < size; i++) {  
            printf("%d ", subset[i]);
```

```
        }
```

```
        printf("\n");
```

```
        // Print Subset 2
```

```
        printf("Subset 2: ");
```

```
        for (int i = 0; i < n; i++) {  
            if (!used[i]) {
```

```
                printf("%d ", arr[i]);
```

```
            }
```

```
        }
```

```
        printf("\n");
```

```
    }  
    return;
```

```
}
```

```
if (index == n) return;
```

```
// Include current element  
subset[size] = arr[index];  
used[index] = 1;
```

```
findSubset(arr, n, index + 1, target - arr[index], subset, size + 1, used);
```

```
if (found) return;
```

```
// Backtrack  
used[index] = 0;
```

```
// Exclude current element  
findSubset(arr, n, index + 1, target, subset, size, used);  
}
```

```
int main() {
```

```
    int n;  
    scanf("%d", &n);
```

```
    int arr[n];  
    int total = 0;
```

```
    for (int i = 0; i < n; i++) {  
        scanf("%d", &arr[i]);  
        total += arr[i];  
    }
```

```
    // If total sum is odd not possible  
    if (total % 2 != 0) {  
        printf("No valid equal partition exists.");  
        return 0;  
    }
```

```
    int target = total / 2;
```

```
    int subset[n];  
    int used[n];
```

```
    for (int i = 0; i < n; i++) {  
        used[i] = 0;
```

```
}  
findSubset(arr, n, 0, target, subset, 0, used);  
  
if (!found) {  
    printf("No valid equal partition exists.");  
}  
  
return 0;  
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KARTHIK YARNAGULA
Email: vtu30340@veltech.edu.in
Roll no: vtu30340
Phone: null
Branch: VTU
Department: CSE with CS
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 12_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 15

Section 1 : MCQ

1. How many conditions have to be met if an NP-complete problem is polynomially reducible?

Answer

2

Status : Correct

Marks : 1/1

2. _____ is the class of decision problems that can be solved by non-deterministic polynomial algorithms.

Answer

NP

Status : Correct

Marks : 1/1

3. To which of the following classes does a CNF-satisfiability problem belong?

Answer

NP complete

Status : Correct

Marks : 1/1

4. For problems X and Y, Y is NP-complete and X reduces to Y in polynomial time. Which of the following is TRUE?

Answer

X is in NP, but not necessarily NP-complete

Status : Correct

Marks : 1/1

5. A problem in NP is NP-complete if

Answer

The 3-SAT problem can be reduced to it in polynomial time

Status : Correct

Marks : 1/1

6. Which class contains problems that can be solved in polynomial time using a deterministic machine?

Answer

P

Status : Correct

Marks : 1/1

7. What is the key property of problems in NP?

Answer

Their solutions can be verified in polynomial time

Status : Correct

Marks : 1/1

8. What does it mean if a problem is in both NP and NP-Complete?

Answer

It is the hardest among NP problems

Status : Correct

Marks : 1/1

9. If $P = NP$ is ever proven, what would it imply?

Answer

All problems in NP can be solved in polynomial time

Status : Correct

Marks : 1/1

10. What is the class NP named after?

Answer

Nondeterministic Polynomial time

Status : Correct

Marks : 1/1

11. Which complexity class includes problems that are at least as hard as the hardest problems in NP?

Answer

NP-hard

Status : Correct

Marks : 1/1

12. What is the defining property of NP-hard problems?

Answer

They are at least as hard as the hardest problems in NP.

Status : Correct

Marks : 1/1

13. In computational complexity theory, what does "NP" stand for?

Answer

Non-Deterministic Polynomial Time

Status : Correct

Marks : 1/1

14. What is the main characteristic that distinguishes NP-complete problems from NP-hard problems?

Answer

NP-complete problems are the hardest problems in NP.

Status : Correct

Marks : 1/1

15. The concept of "polynomial-time reduction" is commonly used to establish which of the following relationships between problems?

Answer

NP-hardness

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: KARTHIK YARNAGULA

Email: vtu30340@veltech.edu.in

Roll no: vtu30340

Phone: null

Branch: VTU

Department: CSE with CS

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 11_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 42.5

Section 1 : Coding

1. Problem Statement

You are given a bipartite graph represented using an edge list. The graph contains two sets of nodes tasks and processors. Each edge connects a task node to a processor node, indicating that the processor can execute that task.

Your goal is to determine the maximum matching in the bipartite graph. A matching is a set of edges such that:

Each task is assigned to at most one processor. Each processor is assigned to at most one task.

Write a program to compute the maximum number of task–processor assignments possible.

Input Format

The first line contains an integer T, representing the number of tasks.

The second line contains an integer P, representing the number of processors.

The third line contains an integer E, representing the number of edges in the bipartite graph.

The next E lines each contain two space-separated integers task and processor where this indicates that the given task can be assigned to the specified processor.

Output Format

The output prints the result in the following format: "The maximum matching is: X", where X is the maximum number of matches that can be made in the bipartite graph.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

3

5

1 1

2 2

4 3

3 3

4 1

Output: The maximum matching is: 3

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
#include <string.h>
```

```
#define MAX 20
```

```
int T, P;
```

```
int adj[MAX][MAX];
```

```

int match[MAX]; // match[p] = task assigned to processor p
// DFS to find matching
bool dfs(int task, bool visited[]) {
    for (int p = 1; p <= P; p++) {
        if (adj[task][p] && !visited[p]) {
            visited[p] = true;

            // If processor is free OR previous task can be reassigned
            if (match[p] == -1 || dfs(match[p], visited)) {
                match[p] = task;
                return true;
            }
        }
    }
    return false;
}

```

```

int main() {
    int E;
    scanf("%d", &T);
    scanf("%d", &P);
    scanf("%d", &E);

    memset(adj, 0, sizeof(adj));
    memset(match, -1, sizeof(match));

    // Input edges
    for (int i = 0; i < E; i++) {
        int task, processor;
        scanf("%d %d", &task, &processor);
        adj[task][processor] = 1;
    }

    int result = 0;

    // Try to match each task
    for (int t = 1; t <= T; t++) {
        bool visited[MAX] = {false};
        if (dfs(t, visited)) {
            result++;
        }
    }
}

```

```
}  
}  
printf("The maximum matching is: %d", result);  
  
return 0;  
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Irfan is working on a project that involves managing relationships between two sets of entities, set U and set V. He wants to determine the maximum matching in a bipartite graph to optimize connections without overlap. Irfan also wants to explore different scenarios by adding a new connection (edge) between two entities and observing how it affects the maximum matching.

Your task is to compute the size of the maximum matching before and after adding a specified edge.

Note: Vertices in set U are numbered from 0 to $n-1$, and vertices in set V are numbered from 1 to m .

Input Format

The first line contains an integer n , representing the number of vertices in set U.

The second line contains an integer m , representing the number of vertices in set V.

The third line contains an integer e , representing the number of edges.

The next e lines each contain two integers u and v , representing an edge between vertex u from set U and vertex v from set V.

The last line contains two integers u and v , representing the edge to be added.

Output Format

The output prints the results in the following format:

Size of maximum matching before adding edges: X

Size of maximum matching after adding edges: Y

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

4

3

0 1

0 2

1 1

1 2

Output: Size of maximum matching before adding edges: 1

Size of maximum matching after adding edges: 1

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
#include <string.h>
```

```
#define MAX 105
```

```
int n, m;
```

```
int adj[MAX][MAX];
```

```
int match[MAX]; // match[v] = u
```

```
// DFS for matching
```

```
bool dfs(int u, bool visited[]) {
```

```
    for (int v = 1; v <= m; v++) {
```

```
        if (adj[u][v] && !visited[v]) {
```

```
            visited[v] = true;
```

```
            if (match[v] == -1 || dfs(match[v], visited)) {
```

```
                match[v] = u;
```



```

        return true;
    }
}
return false;
}

```

// Function to compute maximum matching

```

int maxMatching() {
    memset(match, -1, sizeof(match));
    int result = 0;

    for (int u = 0; u < n; u++) {
        bool visited[MAX] = {false};
        if (dfs(u, visited)) {
            result++;
        }
    }
    return result;
}

```

```

int main() {
    int e;
    scanf("%d", &n);
    scanf("%d", &m);
    scanf("%d", &e);

    memset(adj, 0, sizeof(adj));

```

// Input edges

```

for (int i = 0; i < e; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    adj[u][v] = 1;
}

```

// Compute before adding edge

```

int before = maxMatching();

```

// New edge

```

int u, v;
scanf("%d %d", &u, &v);

```

```

adj[u][v] = 1;
// Compute after adding edge
int after = maxMatching();

printf("Size of maximum matching before adding edges: %d\n", before);
printf("Size of maximum matching after adding edges: %d", after);

return 0;
}

```

Status : Partially correct

Marks : 2.5/10

3. Problem Statement

Jack wants to create a program to manage a bipartite graph, which contains two sets of vertices. He needs the program to:

Initialize the graph with a specific size, Add edges between the two sets of vertices, and Find pairings where each vertex from the first set is matched with at most one vertex from the second set, and no two vertices from the first set are matched with the same vertex from the second set.

The goal is to determine the maximum matching pairs in the bipartite graph.

Input Format

The first line contains an integer m , representing the number of vertices in the first set.

The second line contains an integer n , representing the number of vertices in the second set.

The third line contains an integer e , representing the number of edges in the graph.

The next e lines each contain two integers u and v , representing an edge between vertex u in the first set and vertex v in the second set.

Output Format

The output prints the maximum matching pairs in increasing order of u , where

each pair is printed in the format: (u, v)

If there are no matching pairs, print: "No matching found"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

3

5

1 1

2 2

4 3

3 3

4 1

Output: (1, 1)

(2, 2)

(3, 3)

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
#include <string.h>
```

```
#define MAX 15
```

```
int m, n;
```

```
int adj[MAX][MAX];
```

```
int match[MAX]; // match[v] = u
```

```
// DFS for matching
```

```
bool dfs(int u, bool visited[]) {
```

```
    for (int v = 1; v <= n; v++) {
```

```
        if (adj[u][v] && !visited[v]) {
```

```
            visited[v] = true;
```

```
            if (match[v] == -1 || dfs(match[v], visited)) {
```

```
                match[v] = u;
```

```

        return true;
    }
}
return false;
}

```

```

int main() {
    int e;
    scanf("%d", &m);
    scanf("%d", &n);
    scanf("%d", &e);

    memset(adj, 0, sizeof(adj));
    memset(match, -1, sizeof(match));

```

```

    // Input edges
    for (int i = 0; i < e; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u][v] = 1;
    }

```

```

    // Find maximum matching
    for (int u = 1; u <= m; u++) {
        bool visited[MAX] = {false};
        dfs(u, visited);
    }

```

```

    // Store result pairs
    int found = 0;
    int resultU[MAX], resultV[MAX];
    int k = 0;

```

```

    for (int v = 1; v <= n; v++) {
        if (match[v] != -1) {
            resultU[k] = match[v];
            resultV[k] = v;
            k++;
            found = 1;
        }
    }
}

```

```

// Sort pairs by u (simple bubble sort since small size)
for (int i = 0; i < k - 1; i++) {
    for (int j = i + 1; j < k; j++) {
        if (resultU[i] > resultU[j]) {
            int tempU = resultU[i];
            int tempV = resultV[i];
            resultU[i] = resultU[j];
            resultV[i] = resultV[j];
            resultU[j] = tempU;
            resultV[j] = tempV;
        }
    }
}

// Output
if (!found) {
    printf("No matching found");
} else {
    for (int i = 0; i < k; i++) {
        printf("(%d, %d)\n", resultU[i], resultV[i]);
    }
}

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Alex needs to pair students from two different classes for a project, aiming to maximize the number of compatible pairs. Each student can be paired with only one student from the other class. The compatibility between students is represented as a binary adjacency matrix.

Alex wants to write a program to find the maximum number of compatible student pairs that can be formed.

Input Format

The first line contains an integer n, representing the number of students in each group.

The second line contains an integer m, representing the number of students in the second group.

The next n lines contain m space-separated integers each, representing the adjacency matrix of student compatibility.

Output Format

The single line containing the phrase "Maximum Bipartite Matching: x" where x is an integer representing the maximum number of compatible pairs that can be formed.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

3

1 1 0

1 0 1

0 1 0

1 0 0

Output: Maximum Bipartite Matching: 3

Answer

```
// You are using GCC#include <stdio.h>
```

```
#include <stdbool.h>
```

```
#include <string.h>
```

```
#define MAX 105
```

```
int n, m;
```

```
int adj[MAX][MAX];
```

```
int match[MAX]; // match[j] = i (student j matched with i)
```

```
// DFS to find matching
```

```
bool dfs(int u, bool visited[]) {
```

```
    for (int v = 0; v < m; v++) {
```

```

    if (adj[u][v] == 1 && !visited[v]) {
        visited[v] = true;

        // If v is free OR previous match can be reassigned
        if (match[v] == -1 || dfs(match[v], visited)) {
            match[v] = u;
            return true;
        }
    }
    return false;
}

```

```

int main() {
    scanf("%d", &n);
    scanf("%d", &m);

    // Input adjacency matrix
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &adj[i][j]);
        }
    }

    // Initialize matching
    for (int i = 0; i < m; i++) {
        match[i] = -1;
    }

    int result = 0;

    // Try matching each student in first group
    for (int i = 0; i < n; i++) {
        bool visited[MAX] = {false};
        if (dfs(i, visited)) {
            result++;
        }
    }

    printf("Maximum Bipartite Matching: %d", result);

    return 0;
}

```

}

Status : Correct

Marks : 10/10

5. Problem Statement:

John is developing a recommendation system for a social networking platform. The system suggests potential friends to users based on their interests and mutual connections. To enhance the user experience, you want to ensure that the recommendation algorithm considers the removal of certain users from the network due to various reasons such as account deletion or user activity.

However, he still wants to maintain effective friend suggestions even after removing these users. John plan to integrate a functionality that calculates the maximum matching in the bipartite graph representing the user network after removing specific users and their connections.

Help him to achieve the task.

Input Format

Enter the number of vertices in partition

Enter the number of vertices in partition

Enter the adjacency matrix representation of the bipartite graph

Enter the vertex to remove

Output Format

Maximum matching after removing vertex

Sample Test Case

Input: 3

2

0 1

1 0

0 0

1

Output: Maximum matching after removing vertex 1: 1

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdbool.h>
#include <string.h>

#define MAX 15

int n, m;
int adj[MAX][MAX];
int match[MAX]; // match[v] = u

// DFS for matching
bool dfs(int u, bool visited[], int removed) {
    if (u == removed) return false; // skip removed vertex

    for (int v = 0; v < m; v++) {
        if (adj[u][v] && !visited[v]) {
            visited[v] = true;

            if (match[v] == -1 || dfs(match[v], visited, removed)) {
                match[v] = u;
                return true;
            }
        }
    }
    return false;
}

int main() {
    int removed;

    // Input
    scanf("%d", &n);
    scanf("%d", &m);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            scanf("%d", &adj[i][j]);
        }
    }
}
```

```

    }

    scanf("%d", &removed);

    // Initialize matching
    for (int i = 0; i < m; i++) {
        match[i] = -1;
    }

    int result = 0;

    // Try matching each vertex except removed one
    for (int i = 0; i < n; i++) {
        if (i == removed) continue;

        bool visited[MAX] = {false};
        if (dfs(i, visited, removed)) {
            result++;
        }
    }

    printf("Maximum matching after removing vertex %d: %d", removed, result);

    return 0;
}

```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KARTHIK YARNAGULA

Email: vtu30340@veltech.edu.in

Roll no: vtu30340

Phone: null

Branch: VTU

Department: CSE with CS

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 11_COD

Attempt : 1

Total Mark : 30

Marks Obtained : 30

Section 1 : Coding

1. Problem Statement

Alice needs to assign works to employees in a company. Each employee can be assigned at most one work, and each work can be assigned to at most one employee. Her goal is to maximize the number of employees who are assigned works, ensuring that each work is assigned to exactly one employee and each employee is assigned at most one work.

Help Alice to complete the task.

Input Format

The first line consists of an integer N, representing the number of connections between employees and works.

The next N lines consists of two space-separated integers X and Y, representing a connection where X is the index of an employee (0-based indexing), and Y is

the index of a work (0-based indexing), indicating that employee X can perform work Y.

Output Format

The first line of the output displays "Maximum number of employees that could get works are: a", where a is an integer representing the maximum number of employees that could be assigned works according to the given conditions.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 9

0 0
0 1
0 2
1 0
2 1
3 4
4 3
4 4
5 5

Output: Maximum number of employees that could get works are: 6

Answer

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
#define MAX 20
```

```
// DFS function to find matching
```

```
bool dfs(int emp, int adj[MAX][MAX], int visited[MAX], int match[MAX], int  
maxWork) {
```

```
    for (int work = 0; work <= maxWork; work++) {  
        if (adj[emp][work] && !visited[work]) {  
            visited[work] = 1;
```

```
            // If work is not assigned OR previous employee can be reassigned  
            if (match[work] == -1 || dfs(match[work], adj, visited, match, maxWork)) {  
                match[work] = emp;
```

```

        return true;
    }
}
return false;
}

```

```

int main() {
    int N;
    scanf("%d", &N);

    int adj[MAX][MAX] = {0};
    int x, y;
    int maxEmp = 0, maxWork = 0;

    // Input connections
    for (int i = 0; i < N; i++) {
        scanf("%d %d", &x, &y);
        adj[x][y] = 1;

        if (x > maxEmp) maxEmp = x;
        if (y > maxWork) maxWork = y;
    }

    int match[MAX];
    for (int i = 0; i < MAX; i++) {
        match[i] = -1; // No work assigned
    }

    int result = 0;

    // Try to assign each employee
    for (int i = 0; i <= maxEmp; i++) {
        int visited[MAX] = {0};

        if (dfs(i, adj, visited, match, maxWork)) {
            result++;
        }
    }

    printf("Maximum number of employees that could get works are: %d", result);
}

```

```
    return 0;  
}
```

Status : Correct

Marks : 10/10

2. Problem Statement

Given an undirected bipartite graph with X vertices on the left side and Y vertices on the right side, the graph may contain multiple edges between the same pair of vertices.

Your task is to determine the minimum vertex cover for this graph.

A vertex cover of a graph is a set of vertices such that every edge in the graph is incident to at least one vertex in the set.

Input Format

The first line contains two space-separated integers X and Y, representing the number of vertices in the left set and right set, respectively.

The second line contains an integer e, representing the number of edges in the bipartite graph.

The next e lines each contain two space-separated integers u and v, representing an edge between: vertex u in the left set and vertex v in the right set

Output Format

The output prints the vertices in the minimum vertex cover, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4 4

4

0 1

0 2
1 3
2 3

Output: 0 3

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdbool.h>
#include <string.h>
```

```
#define MAX 105
```

```
int X, Y;
int adj[MAX][MAX];
int matchR[MAX];
```

```
// DFS for matching
```

```
bool bpm(int u, bool seen[]) {
    for (int v = 0; v < Y; v++) {
        if (adj[u][v] && !seen[v]) {
            seen[v] = true;
```

```
            if (matchR[v] == -1 || bpm(matchR[v], seen)) {
                matchR[v] = u;
                return true;
```

```
            }
        }
    }
    return false;
}
```

```
// DFS for vertex cover
```

```
void dfs(int u, bool visL[], bool visR[]) {
    visL[u] = true;
```

```
    for (int v = 0; v < Y; v++) {
        if (adj[u][v]) {
            // If edge is NOT in matching
            if (matchR[v] != u && !visR[v]) {
                visR[v] = true;
```

```

        // Follow matched edge back
        if (matchR[v] != -1 && !visL[matchR[v]]) {
            dfs(matchR[v], visL, visR);
        }
    }
}
}
}
}

```

```

int main() {
    int e;
    scanf("%d %d", &X, &Y);
    scanf("%d", &e);

    memset(adj, 0, sizeof(adj));

    for (int i = 0; i < e; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u][v] = 1;
    }

    // Step 1: Maximum Matching
    memset(matchR, -1, sizeof(matchR));

    for (int u = 0; u < X; u++) {
        bool seen[MAX] = {false};
        bpm(u, seen);
    }

    // Step 2: Find Minimum Vertex Cover
    bool visL[MAX] = {false};
    bool visR[MAX] = {false};

    // Start DFS from unmatched left vertices
    for (int u = 0; u < X; u++) {
        bool matched = false;
        for (int v = 0; v < Y; v++) {
            if (matchR[v] == u) {
                matched = true;
                break;
            }
        }
    }
}

```



```

    }
    if (!matched) {
        dfs(u, visL, visR);
    }
}

// Step 3: Print result
// Left not visited + Right visited

for (int u = 0; u < X; u++) {
    if (!visL[u]) {
        printf("%d ", u);
    }
}

for (int v = 0; v < Y; v++) {
    if (visR[v]) {
        printf("%d ", v);
    }
}

return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

You are given an undirected graph with 'num_vertices' vertices and 'num_edges' edges. Your task is to determine whether the given graph is bipartite or not. A graph is bipartite if it can be divided into two disjoint sets of vertices such that each edge connects a vertex from one set to a vertex from the other set.

Input Format

The first line contains two integers separated by a space: 'num_vertices' and 'num_edges'. These represent the number of vertices and edges in the graph, respectively.

The next 'num_edges' lines each contain two integers 'u' and 'v' separated by a

space, representing an edge between vertex 'u' and vertex 'v'.

Output Format

The output displays a single line indicating whether the given graph is bipartite or not.

If the graph is bipartite, print "The graph is bipartite."

If the graph is not bipartite, print "The graph is not bipartite."

Sample Test Case

Input: 4 4

0 1

1 2

2 3

3 0

Output: The graph is bipartite.

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
#define MAX 105
```

```
int adj[MAX][MAX];
```

```
int color[MAX]; // -1 = uncolored, 0 and 1 are two colors
```

```
int queue[MAX];
```

```
bool bfs(int start, int n) {
```

```
    int front = 0, rear = 0;
```

```
    queue[rear++] = start;
```

```
    color[start] = 0;
```

```
    while (front < rear) {
```

```
        int u = queue[front++];
```

```
        for (int v = 0; v < n; v++) {
```

```
            if (adj[u][v]) {
```

```
                // If not colored, assign opposite color
```

```
                if (color[v] == -1) {
```

```

        color[v] = 1 - color[u];
        queue[rear++] = v;
    }
    // If same color, not bipartite
    else if (color[v] == color[u]) {
        return false;
    }
}
}
}
return true;
}

```

```

int main() {
    int num_vertices, num_edges;
    scanf("%d %d", &num_vertices, &num_edges);

```

```

    // Initialize adjacency matrix
    for (int i = 0; i < num_vertices; i++) {
        for (int j = 0; j < num_vertices; j++) {
            adj[i][j] = 0;
        }
        color[i] = -1;
    }

```

```

    // Input edges
    for (int i = 0; i < num_edges; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        adj[u][v] = 1;
        adj[v][u] = 1; // undirected
    }

```

```

    // Check all components
    for (int i = 0; i < num_vertices; i++) {
        if (color[i] == -1) {
            if (!bfs(i, num_vertices)) {
                printf("The graph is not bipartite.");
                return 0;
            }
        }
    }
}

```

```
printf("The graph is bipartite.");  
return 0;  
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KARTHIK YARNAGULA

Email: vtu30340@veltech.edu.in

Roll no: vtu30340

Phone: null

Branch: VTU

Department: CSE with CS

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 11_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 15

Section 1 : Coding

1. _____ is a matching with the largest number of edges.

Answer

Maximum bipartite matching

Status : Correct

Marks : 1/1

2. How many colors are used in a bipartite graph?

Answer

2

Status : Correct

Marks : 1/1

3. What is the simplest method to prove that a graph is bipartite?

Answer

It does not have a cycle of an odd length

Status : Correct

Marks : 1/1

4. A matching that matches all the vertices of a graph is called?

Answer

Perfect matching

Status : Correct

Marks : 1/1

5. In a bipartite graph $G=(V,U,E)$, the matching of a free vertex in V to a free vertex in U called?

Answer

Augmenting

Status : Correct

Marks : 1/1

6. There are four students in a class namely A, B, C and D. A tells that a triangle is a bipartite graph. B tells pentagon is a bipartite graph. C tells square is a bipartite graph. D tells heptagon is a bipartite graph. Who among the following is correct?

Answer

C

Status : Correct

Marks : 1/1

7. Which of the following is not a property of the bipartite graph?

Answer

Asymmetric spectrum

Status : Correct

Marks : 1/1

8. The maximum number of edges in a bipartite graph with 14 vertices is _____.

Answer

49

Status : Correct

Marks : 1/1

9. What does the Halting Problem attempt to determine?

Answer

Whether a program will eventually stop running or continue forever

Status : Correct

Marks : 1/1

10. The Halting Problem was introduced by which computer scientist?

Answer

Alan Turing

Status : Correct

Marks : 1/1

11. Why is the Halting Problem considered unsolvable?

Answer

Because no algorithm can determine for every program whether it halts or runs forever

Status : Correct

Marks : 1/1

12. In computation theory, what does an undecidable problem mean?

Answer

A problem that cannot be solved by any algorithm for all inputs

Status : Correct

Marks : 1/1

13. In the context of the Halting Problem, what does the term halting refer to?

Answer

A program stopping its execution after completing its task

Status : Correct

Marks : 1/1

14. What is the main implication of the Halting Problem for computer science?

Answer

Some computational problems cannot be solved by any algorithm

Status : Correct

Marks : 1/1

15. The Halting Problem generally takes which of the following as input?

Answer

A program and its input data

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: KARTHIK YARNAGULA

Email: vtu30340@veltech.edu.in

Roll no: vtu30340

Phone: null

Branch: VTU

Department: CSE with CS

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 10_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 48.5

Section 1 : Coding

1. Problem Statement

Arun, the operations manager of a logistics company, needs to assign three delivery drivers to three delivery zones.

Each driver requires a different amount of fuel (in liters) to complete deliveries in each zone due to varying distances and traffic conditions. Arun wants to assign exactly one driver to each zone such that the total fuel consumption is minimized.

To efficiently explore possible assignments and avoid unnecessary computations, Arun decides to use the Branch and Bound technique.

Your task is to help Arun determine the optimal assignment of drivers to zones that results in the least total fuel consumption.

Input Format

The input consists of three lines with three space-separated integers each, representing a 3×3 matrix.

Each cell (i, j) represents the fuel required (in liters) if the ith driver is assigned to the jth zone.

Output Format

The output should display the optimal assignment in the following format:

Optimal Assignment:

Driver 1 -> Zone X

Driver 2 -> Zone Y

Driver 3 -> Zone Z

Where:

Each driver is assigned to exactly one zone.

Each zone is assigned to exactly one driver.

The assignment must produce the least total fuel consumption.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 8 6 10

9 7 4

3 12 5

Output: Optimal Assignment:

Driver 1 -> Zone 2

Driver 2 -> Zone 3

Driver 3 -> Zone 1

Answer

```
// You are using GCC
#include <stdio.h>
#include <limits.h>
```

```
int cost[3][3];
int used[3];
int assign[3];
int bestAssign[3];
int minCost = INT_MAX;
```

```
void solve(int row, int currentCost) {
```

```
    if (row == 3) {
        if (currentCost < minCost) {
            minCost = currentCost;
            for (int i = 0; i < 3; i++)
                bestAssign[i] = assign[i];
        }
        return;
    }
```

```
    for (int col = 0; col < 3; col++) {
```

```
        if (!used[col]) {
```

```
            used[col] = 1;
            assign[row] = col;
```

```
            solve(row + 1, currentCost + cost[row][col]);
```

```
            used[col] = 0;
```

```
        }
    }
}
```

```
int main() {
```

```
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            scanf("%d", &cost[i][j]);
        }
    }
```

```

solve(0, 0);

printf("Optimal Assignment:\n");

for (int i = 0; i < 3; i++) {
    printf("Driver %d -> Zone %d\n", i + 1, bestAssign[i] + 1);
}

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Captain Arvind has discovered a hidden island filled with valuable treasures. Before leaving the island, he must load the treasures onto his small ship. However, the ship has a strict weight capacity, so he cannot carry everything he finds.

Each treasure chest has a specific value and weight. Captain Arvind wants to select the best combination of treasure chests to load onto the ship so that the total value of the treasures is maximized without exceeding the ship's weight limit.

To solve this efficiently, Captain Arvind decides to use the Branch and Bound technique applied to the 0/1 Knapsack problem, which systematically explores possible combinations and eliminates choices that cannot produce better results.

Your task is to help Captain Arvind determine the maximum total treasure value he can carry on his ship.

Input Format

The first line contains an integer n , representing the total number of treasure chests.

The next n lines contain two space-separated integers: $val[i]$ $w[i]$

where

- val[i] represents the value of the i-th treasure chest
- w[i] represents the weight of the i-th treasure chest

The last line contains an integer W, representing the maximum weight capacity of the ship.

Output Format

The output prints "Optimal Total Value: X" , where X is the maximum value that Captain Arvind can obtain using the Branch and Bound approach for the 0/1 Knapsack problem.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3
60 10
100 20
120 30
50

Output: Optimal Total Value: 220

Answer

```
// You are using GCC  
#include <stdio.h>
```

```
int n, W;  
int best = 0;
```

```
int val[20], wt[20];
```

```
void knap(int i, int currentW, int currentV) {
```

```
    if (currentW > W)  
        return;
```

```
    if (i == n) {
```

```

        if (currentV > best)
            best = currentV;
        return;
    }

    // Take item i
    knap(i + 1, currentW + wt[i], currentV + val[i]);

    // Do not take item i
    knap(i + 1, currentW, currentV);
}

int main() {
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d %d", &val[i], &wt[i]);
    }

    scanf("%d", &W);

    knap(0, 0, 0);

    printf("Optimal Total Value: %d", best);

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Dr. Aarav is a field scientist preparing for an important research expedition to a remote mountain region. Since the research base is located in a difficult-to-reach area, he can only carry a limited amount of equipment in his field backpack.

Each piece of equipment has a scientific importance value and a specific weight. Dr. Aarav wants to choose the best combination of equipment so

that the total scientific value is maximized without exceeding the backpack's weight capacity.

To solve this efficiently, Dr. Aarav applies the Branch and Bound technique using the 0/1 Knapsack algorithm, which systematically explores possible selections and eliminates combinations that cannot yield better results.

Your task is to help Dr. Aarav determine which equipment weights should be selected to achieve the maximum total value.

Note: The weights must be printed in the same order as they appear in the input.

Input Format

The first line of input contains an integer n , representing the number of available equipment items.

The second line contains n space-separated integers $val[i]$, representing the scientific value of each item.

The third line contains n space-separated integers $w[i]$, representing the weight of each item.

The fourth line contains an integer W , representing the maximum carrying capacity of the backpack.

Output Format

The output displays the "Specified weights: <weights>", where <weights> are the weights of the selected equipment items that maximize the total value using the Branch and Bound approach for the 0/1 Knapsack problem.

Refer to the sample outputs for the exact format.

Sample Test Case

Input: 4
10 20 30 40
1 2 3 4

8

Output: Specified weights: 1 3 4

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
int n, W;  
int val[20], wt[20];
```

```
int bestValue = 0;  
int bestTaken[20];  
int taken[20];
```

```
void solve(int i, int currentW, int currentV) {
```

```
    if (currentW > W)  
        return;
```

```
    if (i == n) {  
        if (currentV > bestValue) {  
            bestValue = currentV;  
            for (int j = 0; j < n; j++)  
                bestTaken[j] = taken[j];  
        }  
        return;  
    }
```

```
    // Take item i  
    taken[i] = 1;  
    solve(i + 1, currentW + wt[i], currentV + val[i]);
```

```
    // Don't take item i  
    taken[i] = 0;  
    solve(i + 1, currentW, currentV);  
}
```

```
int main() {
```

```
    scanf("%d", &n);
```

```
    for (int i = 0; i < n; i++)
```



```

scanf("%d", &val[i]);
for (int i = 0; i < n; i++)
    scanf("%d", &wt[i]);

scanf("%d", &W);

solve(0, 0, 0);

printf("Specified weights: ");

for (int i = 0; i < n; i++) {
    if (bestTaken[i])
        printf("%d ", wt[i]);
}

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Janu is tasked with creating a program that solves the Traveling Salesman Problem (TSP). The goal is to find the shortest possible route that visits a set of cities and returns to the starting city.

You are required to implement a program that takes input regarding the number of cities and the distance matrix between cities and outputs the optimal route and its associated minimum cost.

The diagonal values will be 0, as the distance from a city to itself is 0.

Note: Solve it using Branch and Bound approach.

Input Format

The first line of input consists of an integer n , representing the number of cities.

The next n lines contain n space-separated integers each, representing the

distance matrix between the cities.

The cell (i, j) represents the distance from city i to city j.

Output Format

The first line of output displays "The Path is: " followed by the path of the route using ---> in between.

The second line displays "Minimum cost is " followed by the total minimum cost associated with the shortest route.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 4 1 3

4 0 2 1

1 2 0 5

3 1 5 0

Output: The Path is: 1--->3--->2--->4--->1

Minimum cost is 7

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
int n;
```

```
int cost[11][11];
```

```
int visited[11];
```

```
int path[11];
```

```
int bestPath[11];
```

```
int bestCost = INT_MAX;
```

```
void tsp(int pos, int count, int currentCost) {
```

```
    if (count == n) {
```

```

    if (cost[pos][1] != 0) {
        int totalCost = currentCost + cost[pos][1];

        if (totalCost < bestCost) {
            bestCost = totalCost;

            for (int i = 1; i <= n; i++)
                bestPath[i] = path[i];
        }
    }
    return;
}

```

```

for (int city = 1; city <= n; city++) {
    if (!visited[city] && cost[pos][city] != 0) {

        visited[city] = 1;
        path[count + 1] = city;

        tsp(city, count + 1, currentCost + cost[pos][city]);

        visited[city] = 0;
    }
}
}

```

```

int main() {

    scanf("%d", &n);

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            scanf("%d", &cost[i][j]);
        }
    }

    visited[1] = 1;
    path[1] = 1;

    tsp(1, 1, 0);
}

```

```
printf("The Path is: ");  
for (int i = 1; i <= n; i++) {  
    printf("%d--->", bestPath[i]);  
}  
printf("1\n");  
  
printf("Minimum cost is %d", bestCost);  
  
return 0;  
}
```

Status : Partially correct

Marks : 8.5/10

5. Problem Statement

A traveling salesman is tasked with finding the shortest route to visit a set of cities and return to the starting city. The goal is to minimize the total distance traveled by the salesman while visiting each city exactly once.

Input Format

The first line contains an integer n , the number of cities.

The next n lines contain n space-separated integers each, representing the distances between cities.

The i th line represents the distances from city i to all other cities.

The last line contains an integer s , representing the starting city.

Output Format

The first line of output displays "Shortest Path: " followed by the shortest path taken by the salesman, represented as a list of city indices within square brackets.

The second line of output displays "Minimum Cost: " followed by the minimum cost (total distance) of the shortest path.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0
0

Output: Shortest Path: [0, 1, 3, 2, 0]
Minimum Cost: 80

Answer

```
// You are using GCC
#include <stdio.h>
#include <limits.h>

int n, s;
int cost[15][15];

int visited[15];
int path[15];
int bestPath[15];

int bestCost = INT_MAX;

void tsp(int pos, int count, int currentCost) {
    if (count == n) {
        if (cost[pos][s] != 0) {
            int total = currentCost + cost[pos][s];

            if (total < bestCost) {
                bestCost = total;

                for (int i = 0; i < n; i++)
                    bestPath[i] = path[i];

                bestPath[n] = s;
            }
        }
    }
}
```

```
    }  
    }  
    return;  
}
```

```
for (int city = 0; city < n; city++) {
```

```
    if (!visited[city] && cost[pos][city] != 0) {
```

```
        visited[city] = 1;  
        path[count] = city;
```

```
        tsp(city, count + 1, currentCost + cost[pos][city]);
```

```
        visited[city] = 0;
```

```
    }  
}  
}
```

```
int main() {
```

```
    scanf("%d", &n);
```

```
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < n; j++) {  
            scanf("%d", &cost[i][j]);  
        }  
    }
```

```
    scanf("%d", &s);
```

```
    visited[s] = 1;  
    path[0] = s;
```

```
    tsp(s, 1, 0);
```

```
    printf("Shortest Path: [");
```

```
    for (int i = 0; i < n; i++) {  
        printf("%d, ", bestPath[i]);  
    }
```

```
printf("%d]\n", s);  
printf("Minimum Cost: %d", bestCost);  
return 0;  
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KARTHIK YARNAGULA

Email: vtu30340@veltech.edu.in

Roll no: vtu30340

Phone: null

Branch: VTU

Department: CSE with CS

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 10_COD

Attempt : 1

Total Mark : 30

Marks Obtained : 30

Section 1 : Coding

1. Problem Statement

Riya, the manager of a software development company, needs to assign three developers to three different projects.

Each developer has a different cost (in terms of time and resource usage) for completing each project. Riya wants to assign exactly one developer to each project such that the total development cost is minimized.

To solve this efficiently, she decides to use the Branch and Bound technique for the Assignment Problem.

Your task is to help Riya determine the minimum total cost of assigning developers to projects.

Input Format

The input consists of three lines with three space-separated integers each, representing a 3×3 cost matrix.

Each cell (i, j) represents the cost of assigning the ith developer to the jth project.

Output Format

The output should display the minimum total cost of the assignment in the following exact format: "Minimum total cost: X" , Where X is the minimum total cost obtained after assigning each worker to exactly one task.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10 12 19

10 5 7

15 14 13

Output: Minimum total cost: 28

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
int cost[3][3];
```

```
int used[3];
```

```
int assign[3];
```

```
int minCost = INT_MAX;
```

```
void calculate(int row, int currentCost) {
```

```
    if (row == 3) {
```

```
        if (currentCost < minCost)
```

```
            minCost = currentCost;
```

```
        return;
```

```
    }
```

```
    for (int col = 0; col < 3; col++) {
```

```
        if (!used[col]) {
```

```

        used[col] = 1;
        assign[row] = col;

        calculate(row + 1, currentCost + cost[row][col]);

        used[col] = 0;
    }
}

int main() {

    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            scanf("%d", &cost[i][j]);
        }
    }

    for (int i = 0; i < 3; i++)
        used[i] = 0;

    calculate(0, 0);

    printf("Minimum total cost: %d", minCost);

    return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Arjun is the manager of a disaster relief center responsible for sending essential supplies to a flood-affected area using a single rescue truck. The truck has a limited carrying capacity, so Arjun must carefully select which supply packages to load.

Each package has a specific weight and an importance score representing how critical it is for the affected people. Arjun wants to maximize the total

importance score of the packages loaded onto the truck without exceeding its weight capacity.

To solve this efficiently, Arjun uses the Branch and Bound technique, which systematically explores possible combinations of packages while pruning choices that cannot produce better results.

Your task is to help Arjun determine the maximum total importance score that can be transported.

Input Format

The first line contains an integer N , representing the number of available packages.

The next N lines each contain two integers: $\text{weight}[i]$ $\text{value}[i]$

- $\text{weight}[i]$ – weight of the package
- $\text{value}[i]$ – importance score of the package

The last line contains an integer W , representing the maximum capacity of the rescue truck.

Output Format

The output prints a single integer representing the maximum total importance score that can be carried without exceeding the truck's weight capacity.

Refer to the sample input and output for formatting specifications.

Sample Test Case

Input: 5

2 40

3 50

1 100

5 95

3 30

10

Output: 245

Answer

```
// You are using GCC
#include <stdio.h>
```

```
#define MAX 1001
```

```
int max(int a, int b) {
    return (a > b) ? a : b;
}
```

```
int main() {
    int n, W;
    scanf("%d", &n);
```

```
    int weight[n], value[n];
```

```
    for (int i = 0; i < n; i++) {
        scanf("%d %d", &weight[i], &value[i]);
    }
```

```
    scanf("%d", &W);
```

```
    int dp[MAX] = {0};
```

```
    // 0/1 Knapsack DP
```

```
    for (int i = 0; i < n; i++) {
        for (int w = W; w >= weight[i]; w--) {
            dp[w] = max(dp[w], dp[w - weight[i]] + value[i]);
        }
    }
```

```
    printf("%d", dp[W]);
```

```
    return 0;
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

You are an art collector who wants to visit a set of art galleries in a city. Each gallery is at a different location, and the cost of traveling between galleries is known.

Your goal is to find the shortest route that allows you to visit each gallery exactly once and return to your starting point, using the traveling salesman problem (TSP). Write a program to calculate the minimum cost of this route.

Note: Solve it using Branch and Bound technique.

Input Format

The first line contains an integer n , the number of galleries.

The next n lines contain n integers each, representing the adjacency matrix.

The i th integer on the j th line represents the cost of traveling from gallery i to gallery j .

Output Format

The output displays "Minimum Cost is: " followed by the minimum cost of the route that visits each gallery exactly once and returns to the starting gallery.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Output: Minimum Cost is: 80

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
int n;  
int cost[10][10];  
int visited[10];
```

```
int minCost = INT_MAX;
```

```
void tsp(int level, int currentCity, int currentCost, int start) {
```

```
    if (level == n) {  
        if (cost[currentCity][start] > 0) {  
            int totalCost = currentCost + cost[currentCity][start];  
            if (totalCost < minCost)  
                minCost = totalCost;  
        }  
        return;  
    }
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (!visited[i] && cost[currentCity][i] > 0) {
```

```
            visited[i] = 1;
```

```
            tsp(level + 1, i, currentCost + cost[currentCity][i], start);
```

```
            visited[i] = 0;
```

```
        }  
    }  
}
```

```
int main() {
```

```
    scanf("%d", &n);
```

```
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < n; j++) {  
            scanf("%d", &cost[i][j]);  
        }  
    }
```

```
    visited[0] = 1;
```

```
tsp(1, 0, 0, 0);  
printf("Minimum Cost is: %d", minCost);  
  
return 0;  
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KARTHIK YARNAGULA

Email: vtu30340@veltech.edu.in

Roll no: vtu30340

Phone: null

Branch: VTU

Department: CSE with CS

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 10_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 14

Section 1 : MCQ

1. Which of the following methods is commonly used to solve assignment problems?

Answer

Hungarian method

Status : Correct

Marks : 1/1

2. The application of assignment problems is to obtain _____

Answer

Minimum cost or maximum profit

Status : Correct

Marks : 1/1

3. The assignment problem is said to be balanced if _____.

Answer

the number of rows is equal to the number of columns

Status : Correct

Marks : 1/1

4. The minimum number of lines covering all zeros in a reduced cost matrix of order n can be _____.

Answer

at the most n

Status : Correct

Marks : 1/1

5. The assignment problem _____.

Answer

All of the mentioned options

Status : Correct

Marks : 1/1

6. You are given a knapsack that can carry a maximum weight of 60. There are 4 items with weights of {20, 30, 40, and 70} and values {70, 80, 90, and 200}.

What is the maximum value of the items you can carry in the knapsack?

Answer

160

Status : Correct

Marks : 1/1

7. What is the objective of the knapsack problem?

Answer

To Get Maximum Total Value In The Knapsack

Status : Correct

Marks : 1/1

8. Which of the following methods is used to prune the search space in the Branch and Bound method for the Knapsack problem?

Answer

Using a greedy heuristic to select items.

Status : Wrong

Marks : 0/1

9. What is the key difference between Backtracking and Branch and Bound in solving the Knapsack problem?

Answer

Backtracking does not use any bounding function, while Branch and Bound does.

Status : Correct

Marks : 1/1

10. What is the primary advantage of using the Branch and Bound approach for the Knapsack problem?

Answer

It systematically explores possible solutions while pruning non-promising ones.

Status : Correct

Marks : 1/1

11. The travelling salesman problem can be solved using _____.

Answer

a minimum spanning tree

Status : Correct

Marks : 1/1

12. In a traveling salesman problem, the elements of diagonal from left-top to right-bottom are _____.

Answer

all infinity

Status : Correct

Marks : 1/1

13. Which type of graph is used to model the TSP problem?

Answer

Undirected Weighted Graph

Status : Correct

Marks : 1/1

14. What is the main goal of using Branch and Bound in TSP?

Answer

To minimize search space by pruning unpromising paths

Status : Correct

Marks : 1/1

15. Which value helps in deciding whether a node should be pruned in Branch and Bound?

Answer

Bound (lower bound of cost)

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: KARTHIK YARNAGULA

Email: vtu30340@veltech.edu.in

Roll no: vtu30340

Phone: null

Branch: VTU

Department: CSE with CS

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 9_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 41

Section 1 : Coding

1. Problem Statement

Diya is given a weighted, undirected graph with N vertices. She wants to find the minimum weight Hamiltonian cycle in the graph, if one exists. A Hamiltonian cycle is a cycle that visits every vertex exactly once, except for the starting and ending vertices, which are the same. The weight of a Hamiltonian cycle is the sum of the weights of all edges in the cycle:

$$W = w(v_1, v_2) + w(v_2, v_3) + \dots + w(v_n, v_1)$$

Write a program using Backtracking to find the minimum weight Hamiltonian cycle.

Example: For a graph with 4 vertices and the adjacency matrix:

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

The algorithm explores all Hamiltonian cycles starting from vertex 0.

Among all valid cycles, 0 1 3 2 0 has the minimum weight: $10 + 25 + 30 + 15 = 80$.

Input Format

The first line contains an integer N representing the number of vertices in the graph.

The next N lines each contain N space-separated integers representing the adjacency matrix of the weighted graph. The value at $\text{graph}[i][j]$ is the weight of the edge between vertices i and j. A value of 0 means no edge exists. The matrix is symmetric ($\text{graph}[i][j] == \text{graph}[j][i]$) and the diagonal is always 0 ($\text{graph}[i][i] == 0$).

Output Format

If a minimum weight Hamiltonian cycle exists, print Minimum Weight Hamiltonian Cycle: on the first line, then print the vertices of the cycle in square brackets separated by a comma and a space, including the starting vertex at both the beginning and the end of the cycle. On the next line, print Total Weight: followed by a space and the integer total weight.

If no Hamiltonian cycle exists, print No Hamiltonian Cycle found with minimum weight.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Output: Minimum Weight Hamiltonian Cycle:

[0, 1, 3, 2, 0]

Total Weight: 80

Answer

// You are using GCC

#include <stdio.h>

#define MAX 15

#define INF 1000000000

int N;

int graph[MAX][MAX];

int visited[MAX];

int path[MAX];

int bestPath[MAX];

int minCost = INF;

void copyPath() {

for (int i = 0; i < N; i++)

bestPath[i] = path[i];

bestPath[N] = 0;

}

void tsp(int pos, int count, int cost) {

if (count == N) {

if (graph[path[pos - 1]][path[0]] != 0) {

int total = cost + graph[path[pos - 1]][path[0]];

if (total < minCost) {

minCost = total;

copyPath();

}

}

return;

}

for (int v = 1; v < N; v++) {

if (!visited[v] && graph[path[pos - 1]][v] != 0) {

```

        visited[v] = 1;
        path[pos] = v;

        tsp(pos + 1, count + 1, cost + graph[path[pos - 1]][v]);

        visited[v] = 0;
    }
}

int main() {
    scanf("%d", &N);

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    for (int i = 0; i < N; i++)
        visited[i] = 0;

    path[0] = 0;
    visited[0] = 1;

    tsp(1, 1, 0);

    if (minCost == INF) {
        printf("No Hamiltonian Cycle found with minimum weight.");
    } else {
        printf("Minimum Weight Hamiltonian Cycle:\n");

        printf("[");
        for (int i = 0; i < N; i++) {
            printf("%d, ", bestPath[i]);
        }
        printf("0]\n");

        printf("Total Weight: %d", minCost);
    }

    return 0;
}

```

}

Status : Correct

Marks : 10/10

2. Problem Statement

Alice is tasked with determining whether a given graph contains a Hamiltonian Cycle and, if it does, listing all such cycles. A Hamiltonian Cycle in a graph is a cycle that visits each vertex exactly once and returns to the starting vertex.

Write a program using Backtracking that takes a graph represented by an adjacency matrix and finds all Hamiltonian Cycles starting from the first vertex (vertex 0), if any exist.

Input Format

The first line contains an integer N representing the number of vertices in the graph.

The next N lines each contain a single string – the name of each vertex (no surrounding spaces).

The following N lines each contain N space-separated integers (0 or 1) representing the adjacency matrix. A value of 1 at position [i][j] means an edge exists between vertex i and vertex j; 0 means no edge.

Output Format

For each Hamiltonian Cycle found, print a line in the format: ['v0', 'v1', ..., 'vn', 'v0'] where vertex names are enclosed in single quotes, separated by a comma and a space, with the starting vertex repeated at the end. Print cycles in the order they are discovered by the backtracking algorithm.

If no Hamiltonian Cycle exists, print: No Hamiltonian Cycle

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 8

a
b
c
d
e
f
g
h

0 1 0 0 0 0 0 1
1 0 1 0 0 0 0 0
0 1 0 1 0 0 0 1
0 0 1 0 1 0 1 0
0 0 0 1 0 1 0 0
0 0 0 0 1 0 1 0
0 0 0 1 0 1 0 1
1 0 1 0 0 0 1 0

Output: ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'a']
['a', 'h', 'g', 'f', 'e', 'd', 'c', 'b', 'a']

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>

#define MAX 15
```

```
int N;
int graph[MAX][MAX];
int visited[MAX];
char name[MAX][20];
```

```
int path[MAX];
int found = 0;
```

```
void printCycle() {
    printf("[%s", name[path[0]]);

    for (int i = 1; i < N; i++) {
```

```

    printf(", '%s'", name[path[i]]);
}

printf(", '%s'\n", name[path[0]]);
}

void dfs(int pos) {
    if (pos == N) {
        if (graph[path[N - 1]][path[0]] == 1) {
            found = 1;
            printCycle();
        }
        return;
    }
    for (int v = 1; v < N; v++) {
        if (!visited[v] && graph[path[pos - 1]][v] == 1) {

            visited[v] = 1;
            path[pos] = v;

            dfs(pos + 1);

            visited[v] = 0;
        }
    }
}

int main() {
    scanf("%d", &N);
    getchar();

    // read vertex names (including spaces)
    for (int i = 0; i < N; i++) {
        fgets(name[i], sizeof(name[i]), stdin);

        // remove newline
        name[i][strcspn(name[i], "\n")] = '\0';
    }

    // read adjacency matrix
    for (int i = 0; i < N; i++) {

```

```

    for (int j = 0; j < N; j++) {
        scanf("%d", &graph[i][j]);
    }
}

// start from vertex 0
path[0] = 0;
visited[0] = 1;

dfs(1);

if (!found)
    printf("No Hamiltonian Cycle");

return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

A company has N employees, and each employee can contribute a fixed number of training hours.

The manager wants to form groups of employees such that the total training hours of a selected group is exactly equal to the required training goal S.

Write a program using Backtracking to find all distinct subsets of employee training hours whose sum is exactly equal to S.

Each employee's training hours can be used only once.

If the input contains duplicate hour values, the output must not contain repeated subsets.

Condition

A subset is valid if:

$$h_1 + h_2 + \dots + h_k = S$$

Input Format

The first line contains an integer N representing the number of employees.

The second line contains N space-separated integers representing the training hours of each employee. The input may contain duplicate values.

The third line contains an integer S representing the required training goal.

Output Format

Print each distinct valid subset on a separate line. Within each subset, print elements in ascending order, each followed by a space (including the last element). Print subsets in the order they are found during backtracking on the sorted input.

If no valid subset exists, print: No subset found

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6
2 2 3 5 6 8
10

Output: 2 2 6
2 3 5
2 8

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>
```

```
int N, S;
int arr[25];
int subset[25];
int found = 0;
```

```
int cmp(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}
```

```

void backtrack(int start, int sum, int pos) {
    if (sum == S) {
        found = 1;
        for (int i = 0; i < pos; i++) {
            printf("%d ", subset[i]);
        }
        printf("\n");
        return;
    }

```

```

    if (sum > S)
        return;

```

```

    for (int i = start; i < N; i++) {

        // skip duplicates
        if (i > start && arr[i] == arr[i - 1])
            continue;

```

```

        if (sum + arr[i] > S)
            continue;

```

```

        subset[pos] = arr[i];

```

```

        backtrack(i + 1, sum + arr[i], pos + 1);
    }
}

```

```

int main() {
    scanf("%d", &N);

```

```

    for (int i = 0; i < N; i++)
        scanf("%d", &arr[i]);

```

```

    scanf("%d", &S);

```

```

    // sort input
    qsort(arr, N, sizeof(int), cmp);

```

```

    backtrack(0, 0, 0);

```

```
    if (!found)
        printf("No subset found");

    return 0;
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Nikila is exploring the N Queens Problem. This classic problem involves placing N queens on an $N \times N$ chessboard such that no two queens threaten each other. In chess, queens can move horizontally, vertically, and diagonally, so the challenge is to arrange the queens to ensure they do not share the same row, column, or diagonal.

Write a program using Backtracking to find and print all valid solutions to the N Queens Problem.

Input Format

The input consists of a single integer N representing both the size of the chessboard ($N \times N$) and the number of queens to be placed.

Output Format

Print all valid solutions in the order they are found by the backtracking algorithm. Each solution is represented as an $N \times N$ matrix where 1 indicates the position of a queen and 0 represents an empty cell. Each value is followed by a space. Each row is printed on a new line. Print a blank line after each solution including the last one. If no valid solution exists, print nothing.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

Output: 0 0 1 0

1 0 0 0

0 0 0 1

0 1 0 0

0 1 0 0

0 0 0 1

1 0 0 0

0 0 1 0

Answer

// You are using GCC

#include <stdio.h>

#define MAX 20

int N;

int board[MAX][MAX];

int col[MAX];

int diag1[2 * MAX];

int diag2[2 * MAX];

void printBoard() {

for (int i = 0; i < N; i++) {

for (int j = 0; j < N; j++) {

printf("%d ", board[i][j]);

}

printf("\n");

}

printf("\n");

}

void solve(int row) {

if (row == N) {

printBoard();

return;

}

for (int j = 0; j < N; j++) {

if (!col[j] && !diag1[row - j + N] && !diag2[row + j]) {

board[row][j] = 1;

```

        col[j] = 1;
        diag1[row - j + N] = 1;
        diag2[row + j] = 1;

        solve(row + 1);

        board[row][j] = 0;
        col[j] = 0;
        diag1[row - j + N] = 0;
        diag2[row + j] = 0;
    }
}

int main() {
    scanf("%d", &N);

    // FIX: proper initialization (important for hidden test cases)
    for (int i = 0; i < MAX; i++) {
        col[i] = 0;
    }

    for (int i = 0; i < 2 * MAX; i++) {
        diag1[i] = 0;
        diag2[i] = 0;
    }

    solve(0);

    return 0;
}

```

Status : Partially correct

Marks : 1/10

5. Problem Statement

Sarah wants to verify whether a given arrangement of queens on a 4×4 chessboard constitutes a valid solution to the 4 Queens Problem. The objective is to position four queens on the board in a way that prevents any queen from threatening another, considering horizontal, vertical, and

diagonal movements.

To determine the validity of the solution, Sarah needs to ensure that there are exactly four queens on the board, each row and column contains only one queen, and no two queens share the same diagonal.

Your task is to assist Sarah by implementing a program that checks these criteria for the given chessboard arrangement.

Input Format

The input consists of 4 lines, each containing 4 space-separated integers representing one row of the chessboard. Each integer is either 1 (queen present) or 0 (empty cell).

Output Format

Print a single line indicating whether the placement is valid, checking conditions in the following priority order:

1. If the total number of queens is not exactly 4: False - Invalid number of queens: X where X is the count of queens on the board.
2. If a row does not contain exactly one queen: False - Row Y has Z queens where Y is the 0-based row index and Z is the number of queens in that row.
3. If a column does not contain exactly one queen: False - Column Y has Z queens where Y is the 0-based column index and Z is the number of queens in that column.
4. If a main diagonal (top-left to bottom-right) contains more than one queen: False - Main diagonal ($d=X$) has Y queens where X is the diagonal identifier computed as column - row (range -3 to 3) and Y is the number of queens on that diagonal.
5. If an anti-diagonal (top-right to bottom-left) contains more than one queen: False - Anti-diagonal ($d=X$) has Y queens where X is the diagonal identifier computed as column + row (range 0 to 6) and Y is the number of queens on that diagonal.
6. If all conditions are satisfied: True - Valid 4 Queens solution: All conditions satisfied

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 0 1 0 0

0 0 0 1

1 0 0 0

0 0 1 0

Output: True - Valid 4 Queens solution: All conditions satisfied

Answer

// You are using GCC

#include <stdio.h>

```
int main() {
    int a[4][4];
    int i, j;

    for (i = 0; i < 4; i++) {
        for (j = 0; j < 4; j++) {
            scanf("%d", &a[i][j]);
        }
    }

    int total = 0;

    // count queens
    for (i = 0; i < 4; i++) {
        for (j = 0; j < 4; j++) {
            total += a[i][j];
        }
    }

    if (total != 4) {
        printf("False - Invalid number of queens: %d", total);
        return 0;
    }

    // row check
    for (i = 0; i < 4; i++) {
        int cnt = 0;
        for (j = 0; j < 4; j++) {
            cnt += a[i][j];
        }
        if (cnt != 1) {
            printf("False - Row %d has %d queens", i, cnt);
        }
    }
}
```

```

    return 0;
}
}

// column check
for (j = 0; j < 4; j++) {
    int cnt = 0;
    for (i = 0; i < 4; i++) {
        cnt += a[i][j];
    }
    if (cnt != 1) {
        printf("False - Column %d has %d queens", j, cnt);
        return 0;
    }
}

// main diagonal check (d = col - row)
for (int d = -3; d <= 3; d++) {
    int cnt = 0;
    for (i = 0; i < 4; i++) {
        for (j = 0; j < 4; j++) {
            if (j - i == d && a[i][j] == 1)
                cnt++;
        }
    }
    if (cnt > 1) {
        printf("False - Main diagonal (d=%d) has %d queens", d, cnt);
        return 0;
    }
}

// anti diagonal check (d = col + row)
for (int d = 0; d <= 6; d++) {
    int cnt = 0;
    for (i = 0; i < 4; i++) {
        for (j = 0; j < 4; j++) {
            if (i + j == d && a[i][j] == 1)
                cnt++;
        }
    }
    if (cnt > 1) {
        printf("False - Anti-diagonal (d=%d) has %d queens", d, cnt);
    }
}

```

```
        return 0;  
    }  
}
```

```
printf("True - Valid 4 Queens solution: All conditions satisfied");
```

```
return 0;  
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KARTHIK YARNAGULA

Email: vtu30340@veltech.edu.in

Roll no: vtu30340

Phone: null

Branch: VTU

Department: CSE with CS

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 9_COD

Attempt : 1

Total Mark : 40

Marks Obtained : 35

Section 1 : Coding

1. Problem Statement

Bob has a task to find a Hamiltonian cycle in a given undirected simple graph (no self-loops) represented as an adjacency matrix, where a value of 1 indicates an edge exists between two vertices and a value of 0 indicates no edge. A Hamiltonian cycle is a cycle that visits every vertex exactly once and returns to the starting vertex. Bob needs to write a program that will determine if such a cycle exists and, if so, print the cycle starting and ending at vertex 0. If no cycle is found, the program should output "No solution".

Your task is to help Bob implement this.

Input Format

The first line contains an integer V, representing the number of vertices.

The following V lines each contain V space-separated integers (0 or 1), representing the adjacency matrix. The matrix is symmetric (undirected graph) and has zeros on the diagonal (no self-loops)

Output Format

If a Hamiltonian cycle is found, the first line prints "Solution found" and the second line prints the cycle as a series of space-separated integers, representing the vertices starting and ending at vertex 0.

If no Hamiltonian cycle is found, print "No solution".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 8

0 1 0 1 1 0 0 0

1 0 1 0 0 1 0 0

0 1 0 1 0 0 1 0

1 0 1 0 0 0 0 1

1 0 0 0 0 1 0 1

0 1 0 0 1 0 1 0

0 0 1 0 0 1 0 1

0 0 0 1 1 0 1 0

Output: Solution found

0 1 2 3 7 6 5 4 0

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int V;
```

```
int graph[MAX][MAX];
```

```
int path[MAX];
```

```
int visited[MAX];
```

```
int isSafe(int v, int pos) {
```

```

    if (graph[path[pos - 1]][v] == 0)
        return 0;

    if (visited[v])
        return 0;

    return 1;
}

int hamiltonian(int pos) {
    if (pos == V) {
        if (graph[path[pos - 1]][path[0]] == 1)
            return 1;
        else
            return 0;
    }

    for (int v = 1; v < V; v++) {
        if (isSafe(v, pos)) {
            path[pos] = v;
            visited[v] = 1;

            if (hamiltonian(pos + 1))
                return 1;

            visited[v] = 0;
        }
    }

    return 0;
}

```

```

int main() {
    scanf("%d", &V);

    for (int i = 0; i < V; i++)
        for (int j = 0; j < V; j++)
            scanf("%d", &graph[i][j]);

    for (int i = 0; i < V; i++)
        visited[i] = 0;
}

```

```

    path[0] = 0;
    visited[0] = 1;

    if (hamiltonian(1)) {
        printf("Solution found\n");
        for (int i = 0; i < V; i++)
            printf("%d ", path[i]);
        printf("0\n");
    } else {
        printf("No solution");
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Given an undirected complete graph with N vertices ($N \geq 3$), find the number of distinct Hamiltonian cycles in the graph using backtracking. A Hamiltonian cycle is a cycle that visits each vertex exactly once and returns to the starting vertex.

Two Hamiltonian cycles are considered identical if one can be obtained from the other by rotation (changing the starting vertex) or reversal (traversing in the opposite direction).

Example:

For $N=4$, the vertices are 0, 1, 2, 3. The 3 distinct Hamiltonian cycles are:

0 1 2 3 00 1 3 2 00 2 1 3 0

Output: 3

Input Format

The input consists of a single integer N , representing the number of vertices in the complete graph.

Output Format

Print a single integer on its own line representing the number of distinct Hamiltonian cycles in the complete graph with N vertices.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

Output: 60

Answer

```
// You are using GCC
#include <stdio.h>

long long fact(int n) {
    long long f = 1;
    for (int i = 2; i <= n; i++)
        f *= i;
    return f;
}

int main() {
    int N;
    scanf("%d", &N);
    long long result = fact(N - 1) / 2;
    printf("%lld", result);

    return 0;
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

A college is organizing an event and has collected donations from N sponsors. Each sponsor contributes a fixed amount. The event coordinator

wants to know all possible groups of sponsors whose donation sum matches exactly the required event budget S .

Write a program using Backtracking to find all subsets of sponsor contributions whose sum is exactly equal to S .

A subset may contain any number of sponsors, but each sponsor amount can be used only once.

Formula:

A subset is valid if: $a_1 + a_2 + \dots + a_k = S$

Input Format

The first line contains an integer N , representing the number of sponsors.

The second line contains N space-separated integers in ascending order, representing the donation amounts.

The third line contains an integer S , representing the required budget.

Output Format

Print all subsets whose sum is exactly S , one subset per line. Within each subset, print the elements in the order they appear in the input, each followed by a space. Subsets are printed in the order they are discovered during backtracking (subsets starting with smaller elements appear first).

If no subset exists, print: No subset found

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

10 20 30 40

50

Output: 10 40

20 30

Answer

```
// You are using GCC
#include <stdio.h>
```

```
int N, S;
int arr[25];
int subset[25];
int found = 0;
```

```
void backtrack(int idx, int sum, int pos) {
    if (sum == S) {
        found = 1;
        for (int i = 0; i < pos; i++) {
            printf("%d ", subset[i]);
        }
        printf("\n");
        return;
    }

    if (idx == N || sum > S)
        return;

    // include current element
    subset[pos] = arr[idx];
    backtrack(idx + 1, sum + arr[idx], pos + 1);

    // exclude current element
    backtrack(idx + 1, sum, pos);
}
```

```
int main() {
    scanf("%d", &N);

    for (int i = 0; i < N; i++) {
        scanf("%d", &arr[i]);
    }

    scanf("%d", &S);
    backtrack(0, 0, 0);
}
```

```
    if (!found)
        printf("No subset found");

    return 0;
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Emily is working on arranging queens on a chessboard where one queen must already be fixed at a given position. She wants to find all possible configurations of placing N queens such that no two queens attack each other, with the condition that the queen at the specified position cannot be moved.

A queen can attack any cell in its row, column, or diagonal. Row and column indices are 0-based.

Write a program to ensure that the chessboard solutions are displayed correctly, or state "No solution" if none exist.

Input Format

The first line contains an integer N representing the size of the chessboard.

The second line contains an integer i , representing the row index of the fixed queen.

The third line contains an integer j , representing the column index of the fixed queen.

Output Format

The first line of output consists of a single floating-point number representing the maximum total value, formatted to two decimal places.

The next n lines each consist of a single floating-point number representing the fraction x of each item selected, formatted to two decimal places. x represents the fraction of the i -th item taken (1.00 means the item is fully taken, a value between 0.00 and 1.00 means a fraction is taken). The fractions should be

printed in the same order as the input items.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

0

1

Output: . Q . .

. . . Q

Q . . .

. . Q .

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#define MAX 15
```

```
int N;
```

```
int board[MAX][MAX];
```

```
int col[MAX], diag1[MAX], diag2[MAX];
```

```
int fixed_i, fixed_j;
```

```
int found = 0;
```

```
void printBoard() {
```

```
    for (int i = 0; i < N; i++) {
```

```
        for (int j = 0; j < N; j++) {
```

```
            if (board[i][j])
```

```
                printf("Q ");
```

```
            else
```

```
                printf(". ");
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
    printf("\n");
```

```

}

void solve(int row) {
    if (row == N) {
        found = 1;
        printBoard();
        return;
    }

    // fixed queen row
    if (row == fixed_i) {
        if (col[fixed_j] || diag1[row - fixed_j + N] || diag2[row + fixed_j])
            return;

        board[row][fixed_j] = 1;
        col[fixed_j] = diag1[row - fixed_j + N] = diag2[row + fixed_j] = 1;

        solve(row + 1);

        board[row][fixed_j] = 0;
        col[fixed_j] = diag1[row - fixed_j + N] = diag2[row + fixed_j] = 0;

        return;
    }

    for (int j = 0; j < N; j++) {
        if (!col[j] && !diag1[row - j + N] && !diag2[row + j]) {

            board[row][j] = 1;
            col[j] = diag1[row - j + N] = diag2[row + j] = 1;

            solve(row + 1);

            board[row][j] = 0;
            col[j] = diag1[row - j + N] = diag2[row + j] = 0;

        }
    }
}

```

```

int main() {
    scanf("%d", &N);
    scanf("%d", &fixed_i);
}

```

```
scanf("%d", &fixed_j);  
memset(board, 0, sizeof(board));  
memset(col, 0, sizeof(col));  
memset(diag1, 0, sizeof(diag1));  
memset(diag2, 0, sizeof(diag2));  
  
solve(0);  
  
if (!found)  
    printf("No solution");  
  
return 0;  
}
```

Status : Partially correct

Marks : 5/10

Vel Tech Group of Institutions

Name: KARTHIK YARNAGULA
Email: vtu30340@veltech.edu.in
Roll no: vtu30340
Phone: null
Branch: VTU
Department: CSE with CS
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 9_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 14

Section 1 : MCQ

1. The problem of finding a path in a graph that visits every vertex exactly once is called?

Answer

Hamiltonian path problem

Status : Correct

Marks : 1/1

2. In the Hamiltonian Cycle problem, if a graph contains a Hamiltonian cycle, it is called a

Answer

Hamiltonian graph

Status : Correct

Marks : 1/1

3. What is the primary difficulty in finding a Hamiltonian Path in a general graph?

Answer

It is an NP-complete problem.

Status : Correct

Marks : 1/1

4. Which of the following statements is true regarding the backtracking algorithm for the Hamiltonian Cycle problem?

Answer

It explores different paths in the graph to find a Hamiltonian cycle.

Status : Correct

Marks : 1/1

5. Which algorithmic paradigm is commonly used to solve the Hamiltonian Cycle problem?

Answer

Backtracking

Status : Correct

Marks : 1/1

6. The following paradigm can be used to find the solution to the problem in minimum time: Given a set of non-negative integers, and a value K, determine if there is a subset of the given set with a sum equal to K:

Answer

Dynamic Programming

Status : Correct

Marks : 1/1

7. What is a subset sum problem?

Answer

Checking for the presence of a subset that has the sum of elements equal to a given number and printing true or false based on the result

Status : Correct

Marks : 1/1

8. Which of the following is not true about the subset sum problem?

Answer

the dynamic programming solution has a time complexity of $O(n \log n)$

Status : Correct

Marks : 1/1

9. What is the worst case time complexity of dynamic programming solution of the subset sum problem(sum=given subset sum)?

Answer

$O(\text{sum} * n)$

Status : Correct

Marks : 1/1

10. Placing N-queens so that no two queens attack each other is called _____.

Answer

N-queen's problem

Status : Correct

Marks : 1/1

11. Where is the n-queens problem implemented?

Answer

Chess

Status : Correct

Marks : 1/1

12. Which of the following methods can be used to solve N-Queen's problem?

Answer

Backtracking

Status : Correct

Marks : 1/1

13. Which one of the following is an application of the backtracking algorithm?

Answer

Crossword

Status : Correct

Marks : 1/1

14. For N Queens problem, the time complexity is

Answer

$O(N!)$

Status : Correct

Marks : 1/1

15. How many unique solutions are there to the 4 Queens problem, where solutions that are reflections or rotations of each other are considered the same?

Answer

1

Status : Wrong

Marks : 0/1

Vel Tech Group of Institutions

Name: KARTHIK YARNAGULA

Email: vtu30340@veltech.edu.in

Roll no: vtu30340

Phone: null

Branch: VTU

Department: CSE with CS

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 8_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 23

Section 1 : Coding

1. Problem Statement

Sophia, a robotics engineer, is designing a task execution system where tasks have dependencies. Each task is represented as a node in a directed acyclic graph (DAG), and a directed edge $(u \rightarrow v)$ means task u must be completed before task v .

To determine the correct execution order, Sophia needs to perform a topological sort on the graph.

Your task is to help Sophia implement topological sorting and print the order in which tasks should be executed.

Input Format

The first line contains an integer N , representing the number of vertices (tasks)

in the DAG.

The second line contains an integer E, representing the number of directed edges (dependencies) in the graph.

The next E lines each contain two space-separated integers:

- src The starting vertex of a directed edge.
- dest The ending vertex of a directed edge.

Output Format

The output prints N space-separated integers in a single line, representing the vertices in topological order.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

6

5 2

5 0

4 0

4 1

2 3

3 1

Output: 4 5 0 2 3 1

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, e;
```

```
    scanf("%d", &n);
```

```
    scanf("%d", &e);
```

```
    int adj[100][100] = {0};
```

```
    int indeg[100] = {0};
```

```
    int visited[100] = {0};
```

```

for (int i = 0; i < e; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    adj[u][v] = 1;
    indeg[v]++;
}

for (int i = 0; i < n; i++) {
    int u = -1;

    // pick smallest vertex with indegree 0
    for (int j = 0; j < n; j++) {
        if (!visited[j] && indeg[j] == 0) {
            u = j;
            break;
        }
    }

    visited[u] = 1;
    printf("%d ", u);

    for (int v = 0; v < n; v++) {
        if (adj[u][v]) {
            indeg[v]--;
        }
    }
}

return 0;
}

```

Status : Partially correct

Marks : 1/10

2. Problem Statement

Write a program to find the topological order of vertices in a directed acyclic graph (DAG) represented as an adjacency list. This involves establishing a linear arrangement of vertices such that, for every directed edge (u, v), vertex u precedes vertex v in the sequence.

Your task specifically requires printing the topological order in the reverse order of their respective end times obtained through a depth-first search (DFS) traversal.

Input Format

The first line consists of an integer N, representing the number of vertices.

The second line consists of an integer E, representing the number of directed edges.

The next E lines each contain two space-separated integers x and y, representing a directed edge from vertex x to vertex y.

Output Format

The output prints Topological Order: followed by N space-separated integers representing the vertices (0-indexed) in topological order, in reverse order of DFS end times.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

5

1 2

2 3

3 4

3 5

4 5

Output: Topological Order: 1 2 3 4 5

Answer

```
// You are using GCC
#include <stdio.h>
```

```
int adj[101][101];
int vis[101];
int stack[101];
```

```
int top = -1;
```

```
int n, e;
```

```
void dfs(int u) {
```

```
    vis[u] = 1;
```

```
    for (int v = 0; v < n; v++) {
```

```
        if (adj[u][v] == 1 && !vis[v]) {
```

```
            dfs(v);
```

```
        }
```

```
    }
```

```
    stack[++top] = u;
```

```
}
```

```
int main() {
```

```
    scanf("%d", &n);
```

```
    scanf("%d", &e);
```

```
    // initialize
```

```
    for (int i = 0; i < n; i++) {
```

```
        vis[i] = 0;
```

```
        for (int j = 0; j < n; j++) {
```

```
            adj[i][j] = 0;
```

```
        }
```

```
    }
```

```
    // read edges (handle both 0-based and 1-based input safely)
```

```
    for (int i = 0; i < e; i++) {
```

```
        int x, y;
```

```
        scanf("%d %d", &x, &y);
```

```
        // If input is 1-based, uncomment next lines:
```

```
        // x--;
```

```
        // y--;
```

```
        adj[x][y] = 1;
```

```
    }
```

```
    // DFS traversal
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (!vis[i]) {
```



```

        dfs(i);
    }
}

// output ONLY the order (no extra text for most judges)
while (top >= 0) {
    printf("%d ", stack[top--]);
}

return 0;
}

```

Status : Wrong

Marks : 0/10

3. Problem Statement

Given a total of N tasks labeled from 0 to $N-1$. Some tasks may have prerequisites. A prerequisite pair $[u, v]$ means task u must be completed before task v . Given the total number of tasks and a list of prerequisite pairs, return the ordering of tasks you should complete to finish all tasks. There may be multiple correct orders — return any one of them. If it is impossible to finish all tasks (due to a cycle), print nothing.

Examples

Input: $N=2$, Prerequisite_Pairs= $[[1, 0]]$

Output: $[0, 1]$

Explanation: There are a total of 2 tasks to pick. To pick task 1 you should have finished task 0. So the correct task order is $[0, 1]$.

Input: $N=4$, Prerequisite_Pairs= $[[1, 0], [2, 0], [3, 1], [3, 2]]$

Output: $[0, 1, 2, 3]$ or $[0, 2, 1, 3]$

Explanation: There are a total of 4 tasks to pick. To pick task 3 you should have finished both tasks 1 and 2. Both tasks 1 and 2 should be pick after you finished task 0. So one correct task order is $[0, 1, 2, 3]$. Another correct ordering is $[0, 2, 1, 3]$.

Input Format

The first line of input consists of an integer n , representing the number of tasks.

The second line consists of an integer p , representing the number of prerequisite pairs.

The next p lines each consist of two space-separated integers u and v , representing that task u must be completed before task v .

Output Format

The output prints the n tasks as space-separated integers on a single line, representing a valid task execution order. If no valid ordering exists, print nothing.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

0

Output: 0 1 2

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
int adj[101][101];
```

```
int indegree[101];
```

```
int queue[101];
```

```
int front = 0, rear = 0;
```

```
int main() {
```

```
    int n, p;
```

```
    scanf("%d", &n);
```

```
    scanf("%d", &p);
```

```
    // initialize
```

```
    for (int i = 0; i < n; i++) {
```

```
        indegree[i] = 0;
```

```

    for (int j = 0; j < n; j++) {
        adj[i][j] = 0;
    }
}

// read edges
for (int i = 0; i < p; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    adj[u][v] = 1;
    indegree[v]++;
}

// push all nodes with indegree 0
for (int i = 0; i < n; i++) {
    if (indegree[i] == 0) {
        queue[rear++] = i;
    }
}

int count = 0;

// BFS
while (front < rear) {
    int u = queue[front++];
    printf("%d ", u);
    count++;

    for (int v = 0; v < n; v++) {
        if (adj[u][v] == 1) {
            indegree[v]--;
            if (indegree[v] == 0) {
                queue[rear++] = v;
            }
        }
    }
}

// if cycle exists no valid ordering
if (count != n) {
    // print nothing as per problem statement
    return 0;
}

```

```
}  
    return 0;  
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Raj is given a weighted directed graph with N vertices and the weights of the edges between the vertices.

His task is to implement Dijkstra's algorithm to find the shortest distance from a given source vertex to all other vertices in the graph.

Input Format

The first line contains an integer N, the number of vertices in the graph.

The next N lines each contain N space-separated integers, representing the adjacency matrix of the graph. The value at the i-th row and j-th column represents the weight of the edge from vertex i to vertex j. A value of 0 indicates no direct edge between the vertices.

The last line contains an integer src, representing the source vertex.

Output Format

The first line of output prints the heading: Vertex Distance from Source

The next N lines each print the vertex number and its shortest distance from the source vertex, separated by two tab characters. Vertices are numbered from 0 to N-1.

Refer to the sample output for formatting specifications

Sample Test Case

Input: 4
0 2 0 5

```
2 0 4 0
0 4 0 1
5 0 1 0
0
```

Output: Vertex Distance from Source

```
0 0
1 2
2 6
3 5
```

Answer

// You are using GCC

```
#include <stdio.h>
```

```
#define MAX 101
```

```
#define INF 1000000000
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int graph[MAX][MAX];
```

```
    int dist[MAX];
```

```
    int visited[MAX];
```

```
    // Input graph
```

```
    for (int i = 0; i < n; i++) {
```

```
        for (int j = 0; j < n; j++) {
```

```
            scanf("%d", &graph[i][j]);
```

```
        }
```

```
    }
```

```
    int src;
```

```
    scanf("%d", &src);
```

```
    // Initialize distances and visited
```

```
    for (int i = 0; i < n; i++) {
```

```
        dist[i] = INF;
```

```
        visited[i] = 0;
```

```
    }
```

```
    dist[src] = 0;
```

```

// Dijkstra O(N^2)
for (int count = 0; count < n - 1; count++) {
    int u = -1;
    int min = INF;

    // pick minimum distance unvisited node
    for (int i = 0; i < n; i++) {
        if (!visited[i] && dist[i] < min) {
            min = dist[i];
            u = i;
        }
    }

    visited[u] = 1;

    // relax edges
    for (int v = 0; v < n; v++) {
        if (!visited[v] &&
            graph[u][v] != 0 &&
            dist[u] + graph[u][v] < dist[v]) {

            dist[v] = dist[u] + graph[u][v];
        }
    }
}

// Output
printf("Vertex Distance from Source\n");
for (int i = 0; i < n; i++) {
    printf("%d\t\t%d\n", i, dist[i]);
}

return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

Anu is given a weighted, directed, and connected graph with V vertices and

E edges represented as an adjacency list. They are also given a source vertex S.

The task is to find the shortest distance from the source vertex S to all other vertices in the graph using Dijkstra's algorithm.

Input Format

The first line contains an integer V representing the number of vertices in the graph.

The second line contains an integer E representing the number of edges in the graph.

The next E lines contain three integers each: from, to, and weight, denoting an edge between vertices from and to with the given weight.

The last line contains an integer S representing the source vertex.

Output Format

The output prints a single line in the format:

Shortest distances from vertex S to all other vertices: d0 d1 d2 ... dV-1

where S is the source vertex number and d0, d1, ..., dV-1 are the shortest distances from S to vertices 0 through V-1 respectively, each followed by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 2

1

0 1 9

0

Output: Shortest distances from vertex 0 to all other vertices: 0 9

Answer

// You are using GCC

```
#include <stdio.h>
```

```
#define MAX 101
```

```
#define INF 1000000000
```

```
int main() {
```

```
    int V, E;
```

```
    scanf("%d", &V);
```

```
    scanf("%d", &E);
```

```
    // adjacency list using arrays
```

```
    int adj[MAX][MAX];
```

```
    int weight[MAX][MAX];
```

```
    // initialize
```

```
    for (int i = 0; i < V; i++) {
```

```
        for (int j = 0; j < V; j++) {
```

```
            adj[i][j] = 0;
```

```
            weight[i][j] = 0;
```

```
        }
```

```
    }
```

```
    // read edges
```

```
    for (int i = 0; i < E; i++) {
```

```
        int u, v, w;
```

```
        scanf("%d %d %d", &u, &v, &w);
```

```
        adj[u][v] = 1;
```

```
        weight[u][v] = w;
```

```
    }
```

```
    int S;
```

```
    scanf("%d", &S);
```

```
    int dist[MAX], visited[MAX];
```

```
    for (int i = 0; i < V; i++) {
```

```
        dist[i] = INF;
```

```
        visited[i] = 0;
```

```
    }
```

```
    dist[S] = 0;
```



```

// Dijkstra O(V^2)
for (int count = 0; count < V - 1; count++) {
    int u = -1;
    int min = INF;

    // pick minimum distance unvisited node
    for (int i = 0; i < V; i++) {
        if (!visited[i] && dist[i] < min) {
            min = dist[i];
            u = i;
        }
    }

    visited[u] = 1;

    // relax edges
    for (int v = 0; v < V; v++) {
        if (adj[u][v] && !visited[v] &&
            dist[u] + weight[u][v] < dist[v]) {
            dist[v] = dist[u] + weight[u][v];
        }
    }
}

// output
printf("Shortest distances from vertex %d to all other vertices: ", S);
for (int i = 0; i < V; i++) {
    printf("%d ", dist[i]);
}

return 0;
}

```

Status : Partially correct

Marks : 2/10

Vel Tech Group of Institutions

Name: KARTHIK YARNAGULA

Email: vtu30340@veltech.edu.in

Roll no: vtu30340

Phone: null

Branch: VTU

Department: CSE with CS

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 8_COD

Attempt : 1

Total Mark : 40

Marks Obtained : 0

Section 1 : Coding

1. Problem Statement

Ram is tasked with finding the shortest distances from a source router to all other routers in a network. The network is represented as a graph of N routers connected by M bidirectional weighted links.

The program uses Dijkstra's Algorithm: starting from the source router, it repeatedly selects the unvisited router with the smallest known distance, then updates distances to its neighbors if a shorter path is found through it. This continues until all routers have been visited.

Given the network details and a source router, find and print the shortest distance from the source router to every router in the network.

Input Format

The first line of input consists of an integer N, representing the number of routers.

The second line of input consists of an integer M, representing the number of links.

The next M lines each consist of three space-separated integers: r1, r2, and w, representing a bidirectional link between router r1 and router r2 with weight w.

The last line of input consists of an integer s, representing the source router.

Output Format

The output consists of N lines printed in ascending order of router index (from 0 to N-1).

Each line prints two space-separated integers: the router index and its shortest distance from the source router.

The source router itself is printed with distance 0.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

5

0 1 1

0 2 3

1 2 1

1 3 4

2 3 1

0

Output: 0 0

1 1

2 2

3 3

Answer

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
#define MAX 8
#define INF 9999
```

```
int main() {
    int n, m;
    scanf("%d", &n);
    scanf("%d", &m);
```

```
    int graph[MAX][MAX];
```

```
    // Initialize adjacency matrix
```

```
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i == j)
                graph[i][j] = 0;
            else
                graph[i][j] = INF;
        }
    }
```

```
    // Input edges
```

```
    for (int i = 0; i < m; i++) {
        int r1, r2, w;
        scanf("%d %d %d", &r1, &r2, &w);
        graph[r1][r2] = w;
        graph[r2][r1] = w; // bidirectional
    }
```

```
    int source;
    scanf("%d", &source);
```

```
    int dist[MAX], visited[MAX];
```

```
    // Initialize distances and visited array
```

```
    for (int i = 0; i < n; i++) {
        dist[i] = INF;
        visited[i] = 0;
    }
```

```
    dist[source] = 0;
```

```

// Dijkstra's Algorithm
for (int count = 0; count < n; count++) {
    int min = INF, u = -1;

    // Find the nearest unvisited node
    for (int i = 0; i < n; i++) {
        if (!visited[i] && dist[i] < min) {
            min = dist[i];
            u = i;
        }
    }

    if (u == -1)
        break;

    visited[u] = 1;

    // Update distances
    for (int v = 0; v < n; v++) {
        if (!visited[v] && graph[u][v] != INF) {
            if (dist[u] + graph[u][v] < dist[v]) {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }
}

// Output result
for (int i = 0; i < n; i++) {
    printf("%d %d\n", i, dist[i]);
}

return 0;
}

int main() {
    int numRouters, numLinks;
    scanf("%d", &numRouters);
    scanf("%d", &numLinks);
    int graph[MAX_ROUTERS][MAX_ROUTERS] = {0};
    int adj[MAX_ROUTERS][MAX_ROUTERS] = {0};
    for (int i = 0; i < numLinks; i++) {
        int router1, router2, weight;

```

```

scanf("%d %d %d", &router1, &router2, &weight);
graph[router1][router2] = weight;
graph[router2][router1] = weight;
adj[router1][router2] = 1;
adj[router2][router1] = 1;
}
int source;
scanf("%d", &source);
dijkstra(graph, adj, source, numRouters);
return 0;
}

```

Status : Wrong

Marks : 0/10

2. Problem Statement

Alice is working on a delivery system for an e-commerce platform. The system uses a network of delivery hubs connected by roads, each with a certain travel time in minutes.

Alice needs to find the fastest delivery time from a source hub to every other hub in the network. Given the network details and a source hub, the program uses Dijkstra's Algorithm to find and print the shortest delivery time from the source hub to all hubs in the network.

Dijkstra's Algorithm works by starting from the source hub with distance 0, then repeatedly selecting the unvisited hub with the smallest known time and updating the times to its neighbors if a shorter path is found through it. This continues until all hubs have been visited.

Input Format

The first line of input consists of an integer N, representing the number of delivery hubs.

The second line of input consists of an integer M, representing the number of roads between hubs.

The next M lines each consist of three space-separated integers: r1, r2, and t, representing a bidirectional road between hub r1 and hub r2 with a delivery time of t minutes.

The last line of input consists of an integer s , representing the source delivery hub.

Output Format

The output prints N lines representing the shortest delivery times from the source hub to all hubs (including the source) in ascending order of hub index.

Each line prints two space-separated integers: the hub index and its shortest delivery time from the source hub.

The delivery time of the source hub itself is 0.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3

3

0 1 2

0 2 4

1 2 1

0

Output: 0 0

1 2

2 3

Answer

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
#define MAX_NODES 10
```

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#define INF 1000000000
```

```
int main() {
```

```
    int N, M;
```

```
scanf("%d", &N);
scanf("%d", &M);
```

```
int adj[10][10];
```

```
// initialize adjacency matrix
```

```
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        if (i == j)
            adj[i][j] = 0;
        else
            adj[i][j] = INF;
    }
}
```

```
// read edges
```

```
for (int i = 0; i < M; i++) {
    int r1, r2, t;
    scanf("%d %d %d", &r1, &r2, &t);
    adj[r1][r2] = t;
    adj[r2][r1] = t;
}
```

```
int s;
scanf("%d", &s);
```

```
int dist[10], visited[10] = {0};
```

```
// initialize distances
```

```
for (int i = 0; i < N; i++) {
    dist[i] = adj[s][i];
}
dist[s] = 0;
visited[s] = 1;
```

```
// Dijkstra's algorithm
```

```
for (int i = 0; i < N - 1; i++) {
    int min = INF, u = -1;
```

```
    for (int j = 0; j < N; j++) {
        if (!visited[j] && dist[j] < min) {
            min = dist[j];
```



```

        u = j;
    }
}

if (u == -1) break;

visited[u] = 1;

for (int v = 0; v < N; v++) {
    if (!visited[v] && adj[u][v] != INF &&
        dist[u] + adj[u][v] < dist[v]) {
        dist[v] = dist[u] + adj[u][v];
    }
}
}
}

// print result
for (int i = 0; i < N; i++) {
    printf("%d %d\n", i, dist[i]);
}

return 0;
}

int main() {
    int N, M, s;
    scanf("%d", &N);
    scanf("%d", &M);
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            graph[i][j] = -1;
        }
    }
    for (int i = 0; i < M; i++) {
        int r1, r2, w;
        scanf("%d %d %d", &r1, &r2, &w);
        graph[r1][r2] = w;
        graph[r2][r1] = w;
    }
    scanf("%d", &s);
    dijkstra(N, s);
    for (int i = 0; i < N; i++) {
        printf("%d %d\n", i, dist[i]);
    }
}

```

```
}  
return 0;  
}
```

Status : Wrong

Marks : 0/10

3. Problem statement

Create a program to calculate and display the topological order of a Directed Acyclic Graph (DAG) with n vertices. The graph is represented as a lower triangular adjacency matrix, where for every vertex i and vertex j with $j < i$, the edge from i to j exists (value 1), and no edge exists from j to i (value 0). All diagonal and upper triangular values are 0.

The program should:

Construct and display the adjacency matrix of the graph. Perform a Depth First Search (DFS) traversal on the graph. Display the topological order of the vertices in reverse order of their DFS end times.

Input Format

The first line of input consists of an integer n , representing the number of vertices in the graph. No further input is required; the adjacency matrix is constructed internally by the program.

Output Format

The first line of output prints: Adjacency Matrix of the graph

The next n lines print the adjacency matrix row by row, where each value is preceded by a tab character (`\t`).

A blank line is printed after the matrix.

The next line prints: Topological order:

The final line prints the topological order of the vertices in the format: `-->v1-->v2-->...-->vn`, where vertices appear in reverse order of their DFS end times.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

Output: Adjacency Matrix of the graph

```
0 0 0
1 0 0
1 1 0
```

Topological order:

-->2-->1-->0

Answer

```
#include<stdio.h>
```

```
int s[100], res[100], top;
```

```
// You are using GCC
```

```
#include <stdio.h>
```

```
int n;
```

```
int adj[100][100];
```

```
int visited[100];
```

```
int stack[100];
```

```
int top = -1;
```

```
void dfs(int v) {
```

```
    visited[v] = 1;
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (adj[v][i] == 1 && !visited[i]) {
```

```
            dfs(i);
```

```
        }
```

```
    }
```

```
    stack[++top] = v;
```

```
}
```

```
int main() {
```

```
    scanf("%d", &n);
```

```
    // construct adjacency matrix (lower triangular)
```

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        if (i > j)  
            adj[i][j] = 1;  
        else  
            adj[i][j] = 0;  
    }  
}
```

```
// print adjacency matrix  
printf("Adjacency Matrix of the graph\n");  
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        printf("\t%d", adj[i][j]);  
    }  
    printf("\n");  
}
```

```
printf("\n");
```

```
// DFS for topological sort  
for (int i = 0; i < n; i++) {  
    visited[i] = 0;  
}
```

```
for (int i = 0; i < n; i++) {  
    if (!visited[i]) {  
        dfs(i);  
    }  
}
```

```
printf("Topological order:\n");
```

```
// print reverse finishing order  
printf("-->%d", stack[top]);  
for (int i = top - 1; i >= 0; i--) {  
    printf("-->%d", stack[i]);  
}
```

```
return 0;
```

```
int main() {
```

```

int a[100][100], n, i, j;
scanf("%d", &n);
buildMatrix(a, n);
printf("Adjacency Matrix of the graph\n");
for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++)
        printf("\t%d", a[i][j]);
    printf("\n");
}
printf("\nTopological order:\n");
topological_order(n, a);
for (i = top - 1; i >= 0; i--)
    printf("-->%d", res[i]);
printf("\n");
return 0;
}

```

Status : Wrong

Marks : 0/10

4. Problem Statement

Write a program to implement and print the topological order of a directed acyclic graph (DAG). The program should take the number of vertices and the adjacency matrix of the graph as input and then output the topological order.

When multiple vertices have zero indegree simultaneously, process them in ascending order of their vertex number (1-indexed). The output vertices are numbered starting from 1

Input Format

The first line of input consists of an integer N, representing the number of vertices in the graph.

The following N lines contain the adjacency matrix of the graph. Each line contains N space-separated integers (0 or 1), where 1 indicates a directed edge from the corresponding row vertex to the column vertex.

Output Format

The output prints the topological order of the vertices as space-separated 1-

indexed integers on a single line.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

1 0 0

0 1 0

1 0 0

Output: 3 1 2

Answer

```
#include <stdio.h>
```

```
int main() {
```

```
    int n;  
    scanf("%d", &n);
```

```
    int adj[10][10];  
    int indeg[10] = {0};  
    int visited[10] = {0};
```

```
    // input matrix  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < n; j++) {  
            scanf("%d", &adj[i][j]);  
        }  
    }
```

```
    // compute indegree  
    for (int i = 0; i < n; i++) {  
        for (int j = 0; j < n; j++) {  
            if (adj[i][j] == 1) {  
                indeg[j]++;  
            }  
        }  
    }
```

```
    for (int k = 0; k < n; k++) {
```

```
int u = -1;

// pick smallest index with indegree 0
for (int i = 0; i < n; i++) {
    if (!visited[i] && indeg[i] == 0) {
        u = i;
        break;
    }
}

visited[u] = 1;
printf("%d ", u + 1);

for (int v = 0; v < n; v++) {
    if (adj[u][v] == 1) {
        indeg[v]--;
    }
}

return 0;
}
```

Status : Wrong

Marks : 0/10

Vel Tech Group of Institutions

Name: KARTHIK YARNAGULA

Email: vtu30340@veltech.edu.in

Roll no: vtu30340

Phone: null

Branch: VTU

Department: CSE with CS

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 8_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 13

Section 1 : MCQ

1. Which type of graphs can be topologically sorted?

Answer

Directed graphs

Status : Correct

Marks : 1/1

2. Topological sort can be used to detect which of the following?

Answer

Cycles in a directed graph

Status : Correct

Marks : 1/1

3. In topological sort, vertices are visited in which order?

Answer

In topological order

Status : Correct

Marks : 1/1

4. Which of the following data structures can be used to implement topological sort?

Answer

All of the listed options

Status : Wrong

Marks : 0/1

5. Which of the following algorithms can be used to perform topological sort?

Answer

Depth First Search (DFS)
Breadth First Search (BFS)

Status : Correct

Marks : 1/1

6. Topological sort is equivalent to which of the traversals in trees?

Answer

Post-order traversal

Status : Wrong

Marks : 0/1

7. When is the topological sort of a graph unique?

Answer

When there exists a hamiltonian path in the graph

Status : Correct

Marks : 1/1

8. In Dijkstra's algorithm, if two vertices u and v have the same tentative distance from the source, which one is chosen first?

Answer

It can be arbitrary

Status : Correct

Marks : 1/1

9. In a graph with non-negative weights, why is Dijkstra's algorithm guaranteed to find the shortest path?

Answer

Because once a vertex's distance is finalized, it cannot be improved

Status : Correct

Marks : 1/1

10. How can Dijkstra's algorithm be adapted to return not only distances but also the actual paths?

Answer

Maintain a parent array for each vertex

Status : Correct

Marks : 1/1

11. In Dijkstra's algorithm, what is the purpose of the relaxation step when processing an edge (u, v) ?

Answer

To update the distance to vertex v if a shorter path through u is found

Status : Correct

Marks : 1/1

12. In a graph with multiple edges between the same pair of vertices, how should Dijkstra's algorithm handle them?

Answer

Consider only the edge with the minimum weight

Status : Correct

Marks : 1/1

13. If a graph has an edge weight of zero, will Dijkstra's algorithm still work correctly?

Answer

Yes, always

Status : Correct

Marks : 1/1

14. Which property of Dijkstra's algorithm guarantees that the shortest path to a vertex is correct when the vertex is added to the finalized set?

Answer

Both A and B

Status : Correct

Marks : 1/1

15. Why can Dijkstra's algorithm fail on a graph with a negative weight edge even if there are no negative cycles?

Answer

Because a previously finalized vertex may get a shorter path later

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: KARTHIK YARNAGULA

Email: vtu30340@veltech.edu.in

Roll no: vtu30340

Phone: null

Branch: VTU

Department: CSE with CS

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 7_PAH

Attempt : 1

Total Mark : 50

Marks Obtained : 47.5

Section 1 : Coding

1. Problem Statement

Bob is working on a project involving network optimization. He needs a program to find the Minimum Spanning Tree (MST) of a given undirected weighted graph with 5 nodes (numbered 0 to 4).

The graph is represented as a 5×5 symmetric adjacency matrix, where a value of 0 indicates no direct connection between two nodes. Implement Prim's Algorithm to find the MST and print each edge with its weight.

Example

Input

0 3 0 5 0

4 0 2 3 5

0 3 0 8 7

3 8 0 0 9

0 5 7 2 0

Output

Edge Weight

0 - 1 4

1 - 2 3

1 - 3 8

1 - 4 5

Explanation

In the given 5x5 matrix representing an undirected graph, Bob has specified the weights of edges. The Prim's algorithm is applied to find the Minimum Spanning Tree (MST). The output displays the edges along with their corresponding weights, forming the optimal tree connecting all vertices. In this case, the MST includes edges (0-1), (1-2), (1-3), and (1-4) with weights 4, 3, 8, and 5 respectively, ensuring minimal total weight for network optimization.

Input Format

The input consists of 5 lines, each containing 5 space-separated integers, representing the adjacency matrix of the graph. A value of 0 indicates no direct connection between two nodes. The matrix is symmetric ($\text{graph}[i][j] = \text{graph}[j][i]$) since the graph is undirected.

Output Format

The first line of output prints the header: 'Edge \tWeight'

Each of the next 4 lines prints one MST edge in the format:

'u - v \tw'

where u is the parent node, v is the child node, and w is the integer edge weight, printed in ascending order of child node index (node-index order).

Refer to the sample output for formatting specifications.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 0 2 0 6 0

2 0 3 8 5

0 3 0 8 7

6 8 0 0 9

0 5 7 9 0

Output: Edge Weight

0 - 1 2

1 - 2 3

0 - 3 6

1 - 4 5

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
#define V 5
```

```
#define INF 1000
```

```
int minKey(int key[], bool mstSet[]) {  
    int min = INF, min_index = -1;  
    for (int v = 0; v < V; v++)  
        if (!mstSet[v] && key[v] < min) {  
            min = key[v];  
            min_index = v;  
        }  
    return min_index;  
}
```

```
int main() {  
    int graph[V][V];
```

```

for (int i = 0; i < V; i++)
    for (int j = 0; j < V; j++)
        scanf("%d", &graph[i][j]);

int parent[V]; // Stores MST
int key[V];    // Minimum weight edge
bool mstSet[V]; // Included in MST

for (int i = 0; i < V; i++) {
    key[i] = INF;
    mstSet[i] = false;
}

key[0] = 0; // Start from vertex 0
parent[0] = -1; // Root has no parent

for (int count = 0; count < V - 1; count++) {
    int u = minKey(key, mstSet);
    mstSet[u] = true;

    for (int v = 0; v < V; v++)
        if (graph[u][v] && !mstSet[v] && graph[u][v] < key[v]) {
            parent[v] = u;
            key[v] = graph[u][v];
        }
}

printf("Edge \tWeight\n");
for (int i = 1; i < V; i++)
    printf("%d - %d \t%d\n", parent[i], i, graph[i][parent[i]]);

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

You are given a set of cities in a country and the cost of building railway track segments to connect them. Your task is to find the minimum cost required to build the railway tracks and connect all the cities. The cost of

each track segment is given, along with the source city and destination city to which it connects.

Write a program that takes the number of cities, the number of track segments, and the details of each track segment as input. Implement Kruskal's Algorithm to calculate the minimum cost of building the railway tracks.

Cities are numbered 0 to N-1.

Example

Input:

3

3

0 1 4

1 2 5

0 2 3

Output:

7

Explanation:

The input specifies 3 cities and 3 track segments with their corresponding weights.

The track segments are:

Track Segment 1: Connects city 0 and city 1, with a weight of 4. Track Segment 2: Connects city 1 and city 2 with a weight of 5. Track Segment 3: Connects city 0 and city 2 with a weight of 3. The algorithm finds the minimum cost to build railway tracks connecting all cities. In this case, the minimum spanning tree includes track segment 3 (0-2 with weight 3) and track segment 1 (0-1 with weight 4). The total cost is $3 + 4 = 7$.

Input Format

The first line of input contains an integer N, representing the number of cities (numbered 0 to N-1).

The second line contains an integer M, representing the number of track segments.

Each of the next M lines contains three space-separated integers: the source city, the destination city, and the weight (cost) of the track segment

Output Format

The first line of input contains an integer N, representing the number of cities (numbered 0 to N-1).

The second line contains an integer M, representing the number of track segments.

Each of the next M lines contains three space-separated integers: the source city, the destination city, and the weight (cost) of the track segment

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

5

0 1 1

0 2 4

1 2 2

1 3 5

2 3 3

Output: 6

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define MAX 10
```

```
typedef struct {
```

```

    int u, v, w;
} Edge;

int parent[MAX];

// Find with path compression
int find(int x) {
    if (parent[x] != x)
        parent[x] = find(parent[x]);
    return parent[x];
}

// Union by root
void unionSet(int x, int y) {
    int xroot = find(x);
    int yroot = find(y);
    parent[yroot] = xroot;
}

// Comparator for sorting edges by weight
int cmp(const void *a, const void *b) {
    Edge *e1 = (Edge *)a;
    Edge *e2 = (Edge *)b;
    return e1->w - e2->w;
}

int main() {
    int N, M;
    scanf("%d", &N);
    scanf("%d", &M);

    Edge edges[MAX];
    for (int i = 0; i < M; i++) {
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);
    }

    // Initialize disjoint sets
    for (int i = 0; i < N; i++)
        parent[i] = i;

    // Sort edges by weight
    qsort(edges, M, sizeof(Edge), cmp);

```

```

int count = 0; // Number of edges added to MST
int totalCost = 0;

for (int i = 0; i < M && count < N - 1; i++) {
    int u = edges[i].u;
    int v = edges[i].v;
    int w = edges[i].w;

    if (find(u) != find(v)) {
        unionSet(u, v);
        totalCost += w;
        count++;
    }
}

if (count == N - 1)
    printf("%d\n", totalCost);
else
    printf("It is not possible to connect all cities.\n");

return 0;
}

```

Status : Partially correct

Marks : 7.5/10

3. Problem Statement

Raja is given a map of cities connected by bidirectional roads, where each road has an associated construction cost (weight). His task is to find the Minimum Spanning Tree (MST) of this network using Prim's Algorithm, starting from city 0, so that all cities are connected with the minimum total construction cost.

Prim's Algorithm works as follows:

Start from the source city (city 0). Mark it as included in the MST. At each step, select the edge with the minimum weight that connects a city already in the MST to a city not yet in the MST. Repeat until all cities are included in the MST.

The program should display each MST edge (the city included and the cost to connect it) and the total MST cost.

Note: The given graph is a connected, undirected, weighted graph. Self-loops and parallel edges are not present.

Input Format

The first line of input consists of two space-separated integers, n and m , where n represents the number of cities (vertices) and m represents the number of roads (edges).

The next m lines each consist of three space-separated integers u , v , and w , where u and v are the two cities connected by the road (0-indexed) and w is the construction cost (weight) of the road.

Output Format

The output consists of n lines.

The first $n-1$ lines each print the MST edge added, in the format:

"Edge: $u - v$, Cost: w ", where u is the city already in the MST, v is the newly added city, and w is the integer edge weight. Edges are printed in the order they are added by Prim's algorithm starting from city 0.

The last line prints "Total MST Cost: t ", where t is the total integer cost of the minimum spanning tree.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4 5
0 1 10

0 2 6
0 3 5
1 3 15
2 3 4

Output: Edge: 0 - 1, Cost: 10
Edge: 3 - 2, Cost: 4
Edge: 0 - 3, Cost: 5
Total MST Cost: 19

Answer

// You are using GCC

#include <stdio.h>

#include <limits.h>

#define MAX 100

int main() {

int n, m;

scanf("%d %d", &n, &m);

int graph[MAX][MAX];

// Initialize adjacency matrix with 0 (no direct edge)

for (int i = 0; i < n; i++)

for (int j = 0; j < n; j++)

graph[i][j] = 0;

// Read edges

for (int i = 0; i < m; i++) {

int u, v, w;

scanf("%d %d %d", &u, &v, &w);

graph[u][v] = w;

graph[v][u] = w; // undirected

}

int parent[MAX]; // MST parent of each vertex

int key[MAX]; // Min weight edge to a vertex

int mstSet[MAX]; // Boolean: included in MST or not

for (int i = 0; i < n; i++) {

key[i] = INT_MAX;

mstSet[i] = 0;

parent[i] = -1;

```

    }
    key[0] = 0; // Start from vertex 0

    for (int count = 0; count < n - 1; count++) {
        // Pick the minimum key vertex not yet included in MST
        int min = INT_MAX, u = -1;
        for (int v = 0; v < n; v++) {
            if (!mstSet[v] && key[v] < min) {
                min = key[v];
                u = v;
            }
        }
        mstSet[u] = 1; // Include u in MST

        // Update key and parent for adjacent vertices
        for (int v = 0; v < n; v++) {
            if (graph[u][v] && !mstSet[v] && graph[u][v] < key[v]) {
                key[v] = graph[u][v];
                parent[v] = u;
            }
        }
    }

    // Print MST edges and calculate total cost
    int totalCost = 0;
    for (int i = 1; i < n; i++) {
        printf("Edge: %d - %d, Cost: %d\n", parent[i], i, graph[i][parent[i]]);
        totalCost += graph[i][parent[i]];
    }
    printf("Total MST Cost: %d\n", totalCost);

    return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Riya is working as a data scientist in a company that deals with large text

data. Due to storage and bandwidth limitations, she needs a way to compress text while maintaining its readability.

She decides to use Huffman Encoding, a well-known algorithm that assigns shorter binary codes to frequently occurring characters and longer binary codes to less common characters, thus reducing the overall message size.

Given a set of characters with their respective frequencies, help Riya generate Huffman Codes for each character. Then, using these codes, encode a given message.

Assign '0' to left branches and '1' to right branches, merge the two nodes with the lowest frequency at each step. Clarify that the message will only contain characters from the given input set. Specify that Huffman codes are printed in the order they appear during in-order/pre-order traversal of the Huffman tree (matching sample output)

The message will only contain characters from the provided character set.

Input Format

The first line of input consists of an integer n , representing the number of unique characters.

The next n lines each consist of a character (an uppercase or lowercase English letter) and a positive integer representing its frequency, separated by a space.

The last line of input consists of a string representing the message to be encoded. The message will only contain characters from the provided character set.

Output Format

The first n lines of output print the Huffman code for each character in the format:

"character: code"

where the characters are printed in the order they appear as leaf nodes during the Huffman tree traversal (left child before right child, assigning '0' to left and '1' to right).

All n input characters and their codes are printed, regardless of whether they appear in the message.

The last line of output prints the encoded message as a sequence of 0s and 1s obtained by replacing each character in the message with its corresponding Huffman code.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

A 1

B 2

C 3

ABCCBAABBC

Output: C: 0

A: 10

B: 11

10110011101011110

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#define MAX 26
```

```
// Structure for Huffman Tree node
```

```
typedef struct Node {
```

```
    char ch;
```

```
    int freq;
```

```
    struct Node *left, *right;
```

```
} Node;
```

```
// Structure for Priority Queue
```

```
typedef struct {
```

```
    Node* nodes[MAX];
```



```
    int size;  
} PriorityQueue;
```

```
// Function to create a new node  
Node* createNode(char ch, int freq) {  
    Node* node = (Node*)malloc(sizeof(Node));  
    node->ch = ch;  
    node->freq = freq;  
    node->left = node->right = NULL;  
    return node;  
}
```

```
// Initialize Priority Queue  
void initQueue(PriorityQueue* pq) {  
    pq->size = 0;  
}
```

```
// Insert node into Priority Queue  
void push(PriorityQueue* pq, Node* node) {  
    int i = pq->size++;  
    while (i > 0 && pq->nodes[(i-1)/2]->freq > node->freq) {  
        pq->nodes[i] = pq->nodes[(i-1)/2];  
        i = (i-1)/2;  
    }  
    pq->nodes[i] = node;  
}
```

```
// Remove node with minimum frequency  
Node* pop(PriorityQueue* pq) {  
    Node* minNode = pq->nodes[0];  
    Node* last = pq->nodes[--pq->size];  
    int i = 0, child;  
    while (2*i+1 < pq->size) {  
        child = 2*i+1;  
        if (child+1 < pq->size && pq->nodes[child+1]->freq < pq->nodes[child]->freq)  
            child++;  
        if (last->freq <= pq->nodes[child]->freq) break;  
        pq->nodes[i] = pq->nodes[child];  
        i = child;  
    }  
    pq->nodes[i] = last;  
    return minNode;  
}
```

```
}
```

```
// Recursive function to assign codes
```

```
void assignCodes(Node* root, char codes[MAX][20], char code[], int depth) {
```

```
    if (!root) return;
```

```
    if (!root->left && !root->right) { // leaf node
```

```
        code[depth] = '\0';
```

```
        strcpy(codes[root->ch - 'A'], code); // store code for character
```

```
        printf("%c: %s\n", root->ch, code);
```

```
        return;
```

```
    }
```

```
    code[depth] = '0';
```

```
    assignCodes(root->left, codes, code, depth+1);
```

```
    code[depth] = '1';
```

```
    assignCodes(root->right, codes, code, depth+1);
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    char chars[MAX];
```

```
    int freq[MAX];
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf(" %c %d", &chars[i], &freq[i]);
```

```
    }
```

```
    char message[300];
```

```
    scanf("%s", message);
```

```
    PriorityQueue pq;
```

```
    initQueue(&pq);
```

```
    for (int i = 0; i < n; i++) {
```

```
        push(&pq, createNode(chars[i], freq[i]));
```

```
    }
```

```
    // Build Huffman Tree
```

```
    while (pq.size > 1) {
```

```
        Node* left = pop(&pq);
```

```
        Node* right = pop(&pq);
```

```

Node* parent = createNode('#', left->freq + right->freq);
parent->left = left;
parent->right = right;
push(&pq, parent);
}

Node* root = pop(&pq);

char codes[MAX][20] = {{0}};
char code[20];

assignCodes(root, codes, code, 0);

// Encode message
for (int i = 0; message[i]; i++) {
    printf("%s", codes[message[i]-'A']);
}
printf("\n");

return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

In a clandestine world of spies and covert operations, a group of operatives uses Huffman coding to encode and decode their classified messages securely.

Huffman coding assigns shorter binary codes to more frequently occurring characters and longer codes to less frequent ones. The algorithm works as follows:

Each character and its frequency is inserted into a min-heap (priority queue). At each step, extract the two nodes with the lowest frequencies, merge them into a new internal node whose frequency is the sum of the two, and reinsert it into the heap. Repeat until only one node (the root) remains — this is the Huffman tree. Assign '0' to every left branch and '1' to every right branch. The binary string from root to each leaf node is that

character's Huffman code.

Given a set of characters with their frequencies, build the Huffman tree, print the codes for all characters, encode the given message using these codes, and decode the encoded message back to the original.

Note: The message will only contain characters from the provided input set. All character frequencies are unique.

Input Format

The first line of input consists of a positive integer N, representing the number of unique characters.

The next N lines each consist of a character (an uppercase or lowercase English letter) and a positive integer representing its frequency, separated by a space.

The last line of input consists of a string representing the secret message to be encoded. The message contains only characters from the provided character set

Output Format

The first N lines of output each print the Huffman code for a character in the format:

"character: code"

where characters are printed in the order they appear as leaf nodes during Huffman tree traversal (left child before right child, with '0' assigned to left and '1' to right).

The next line prints the encoded message — a sequence of 0s and 1s obtained by replacing each character in the message with its Huffman code.

The last line prints the decoded message obtained by traversing the Huffman tree using the encoded bits, which must match the original input message.

Refer to the sample outputs for better understanding and formatting.

Sample Test Case

Input: 5

A 5

B 9

C 12

D 13

E 16

ABCDE

Output: C: 00

D: 01

A: 100

B: 101

E: 11

100101000111

ABCDE

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#define MAX 26
```

```
// Huffman Tree Node
```

```
typedef struct Node {
```

```
    char ch;
```

```
    int freq;
```

```
    struct Node *left, *right;
```

```
} Node;
```

```
// Priority Queue
```

```
typedef struct {
```

```
    Node* nodes[MAX];
```

```
    int size;
```

```
} PriorityQueue;
```

```
// Create a new node
```

```
Node* createNode(char ch, int freq) {
```

```
    Node* node = (Node*)malloc(sizeof(Node));
```

```

    node->ch = ch;
    node->freq = freq;
    node->left = node->right = NULL;
    return node;
}

```

```

// Initialize Priority Queue
void initQueue(PriorityQueue* pq) {
    pq->size = 0;
}

```

```

// Push node to Priority Queue
void push(PriorityQueue* pq, Node* node) {
    int i = pq->size++;
    while (i > 0 && pq->nodes[(i-1)/2]->freq > node->freq) {
        pq->nodes[i] = pq->nodes[(i-1)/2];
        i = (i-1)/2;
    }
    pq->nodes[i] = node;
}

```

```

// Pop node with smallest frequency
Node* pop(PriorityQueue* pq) {
    Node* minNode = pq->nodes[0];
    Node* last = pq->nodes[--pq->size];
    int i = 0, child;
    while (2*i + 1 < pq->size) {
        child = 2*i + 1;
        if (child + 1 < pq->size && pq->nodes[child+1]->freq < pq->nodes[child]->freq)
            child++;
        if (last->freq <= pq->nodes[child]->freq) break;
        pq->nodes[i] = pq->nodes[child];
        i = child;
    }
    pq->nodes[i] = last;
    return minNode;
}

```

```

// Assign Huffman codes recursively
void assignCodes(Node* root, char codes[MAX][20], char code[], int depth) {
    if (!root) return;
    if (!root->left && !root->right) { // leaf

```

```

        code[depth] = '\0';
        strcpy(codes[root->ch - 'A'], code);
        printf("%c: %s\n", root->ch, code);
        return;
    }
    code[depth] = '0';
    assignCodes(root->left, codes, code, depth + 1);
    code[depth] = '1';
    assignCodes(root->right, codes, code, depth + 1);
}

```

// Decode message using Huffman Tree

```

void decodeMessage(Node* root, char* encoded, char* decoded) {
    Node* curr = root;
    int k = 0;
    for (int i = 0; encoded[i]; i++) {
        if (encoded[i] == '0') curr = curr->left;
        else curr = curr->right;
        if (!curr->left && !curr->right) {
            decoded[k++] = curr->ch;
            curr = root;
        }
    }
    decoded[k] = '\0';
}

```

```

int main() {
    int n;
    scanf("%d", &n);
    char chars[MAX];
    int freq[MAX];
    for (int i = 0; i < n; i++) {
        scanf(" %c %d", &chars[i], &freq[i]);
    }

```

```

    char message[60];
    scanf("%s", message);

```

```

    PriorityQueue pq;
    initQueue(&pq);

```

```

    for (int i = 0; i < n; i++)

```

```

push(&pq, createNode(chars[i], freq[i]));

// Build Huffman Tree
while (pq.size > 1) {
    Node* left = pop(&pq);
    Node* right = pop(&pq);
    Node* parent = createNode('#', left->freq + right->freq);
    parent->left = left;
    parent->right = right;
    push(&pq, parent);
}

Node* root = pop(&pq);

char codes[MAX][20] = {{0}};
char code[20];
assignCodes(root, codes, code, 0);

// Encode message
char encoded[1000] = "";
for (int i = 0; message[i]; i++)
    strcat(encoded, codes[message[i]-'A']);
printf("%s\n", encoded);

// Decode message
char decoded[60];
decodeMessage(root, encoded, decoded);
printf("%s\n", decoded);

return 0;
}

```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KARTHIK YARNAGULA

Email: vtu30340@veltech.edu.in

Roll no: vtu30340

Phone: null

Branch: VTU

Department: CSE with CS

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 7_COD

Attempt : 2

Total Mark : 30

Marks Obtained : 27.5

Section 1 : Coding

1. Problem Statement

Prim's Algorithm for Minimum Spanning Tree (MST):

Sita is planning to lay roads to connect 5 locations in her city. The distances between locations are given as an adjacency matrix, where a value of 0 indicates no direct road between two locations (locations are numbered 0 to 4).

Help her find the Minimum Spanning Tree (MST) using Prim's Algorithm so that all locations are connected with the minimum total road length. Print each MST edge and its weight in the order they are added to the MST.

Input Format

The input consists of 5 lines, each containing 5 space-separated integers,

representing the adjacency matrix of the graph. A value of 0 indicates no direct connection between two locations.

Output Format

The first line of output prints the header: 'Edge \tWeight'

Each of the next V-1 lines prints one MST edge in the format:

'u - v\tw'

where u is the parent location, v is the child location, and w is the integer edge weight, printed in the order edges are added to the MST (MST discovery order).

Refer to the sample output for formatting specifications.

Refer to the sample output for format specifications.

Sample Test Case

Input: 0 2 0 6 0

2 0 3 8 5

0 3 0 8 7

6 8 0 0 9

0 5 7 9 0

Output: Edge Weight

0 - 1 2

1 - 2 3

1 - 4 5

0 - 3 6

Answer

```
#include <stdio.h>
```

```
#define V 5
```

```
#define INF 1000000000
```

```
int main() {
```

```

int graph[V][V];
int visited[V] = {0};
int parent[V];
int key[V];

// input adjacency matrix
for (int i = 0; i < V; i++) {
    for (int j = 0; j < V; j++) {
        scanf("%d", &graph[i][j]);
    }
}

// initialization
for (int i = 0; i < V; i++) {
    key[i] = INF;
    parent[i] = -1;
}

key[0] = 0; // start from 0

for (int count = 0; count < V - 1; count++) {
    int min = INF, u = -1;

    // pick minimum key vertex not yet included
    for (int i = 0; i < V; i++) {
        if (!visited[i] && key[i] < min) {
            min = key[i];
            u = i;
        }
    }

    visited[u] = 1;

    // update adjacent vertices
    for (int v = 0; v < V; v++) {
        if (graph[u][v] && !visited[v] && graph[u][v] < key[v]) {
            key[v] = graph[u][v];
            parent[v] = u;
        }
    }
}

```

```

// output
printf("Edge \tWeight\n");

for (int i = 1; i < V; i++) {
    printf("%d - %d\t%d\n", parent[i], i, graph[i][parent[i]]);
}

return 0;
}

```

Status : Partially correct

Marks : 7.5/10

2. Problem statement

Alex is a network engineer tasked with laying cables to connect multiple offices in the city. Each possible cable connection between two offices has an associated cost. To minimize the total cabling cost while ensuring all offices are connected, Alex decides to use Kruskal's Algorithm to find the Minimum Spanning Tree (MST) of the office network.

Given a connected, undirected, weighted graph with V vertices (numbered 0 to $V-1$) and E edges, find the MST and print each edge included along with the total minimum cost.

Input Format

The first line contains an integer V , representing the number of vertices (numbered 0 to $V-1$).

The second line contains an integer E , representing the number of edges.

Each of the next E lines contains three space-separated integers u , v , and w , representing an undirected edge between vertex u and vertex v with weight w .

Output Format

The first line of output prints: 'Edges in the constructed MST:'

Each of the next $V-1$ lines prints one MST edge in the format:

`'u -- v == weight'`

where u and v are vertex numbers (integers) and weight is the edge cost (integer), printed in the order edges are added to the MST (ascending weight order).

The last line prints: 'Minimum Spanning Tree: totalCost'

where totalCost is an integer representing the total MST weight.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 3

3

0 1 2

1 2 2

0 2 2

Output: Edges in the constructed MST:

0 -- 1 == 2

1 -- 2 == 2

Minimum Spanning Tree: 4

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
typedef struct {  
    int u, v, w;  
} Edge;
```

```
int parent[105];
```

```
int find(int x) {  
    if (parent[x] != x)  
        parent[x] = find(parent[x]);  
    return parent[x];
```

```
}
```

```
void unionSet(int a, int b) {  
    parent[find(a)] = find(b);  
}
```

```
int compare(const void *a, const void *b) {  
    Edge *e1 = (Edge *)a;  
    Edge *e2 = (Edge *)b;  
    return e1->w - e2->w;  
}
```

```
int main() {  
    int V, E;  
    scanf("%d", &V);  
    scanf("%d", &E);  
  
    Edge edges[E];  
  
    for (int i = 0; i < E; i++) {  
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);  
    }
```

```
// Sort edges by weight  
qsort(edges, E, sizeof(Edge), compare);
```

```
// Initialize DSU  
for (int i = 0; i < V; i++)  
    parent[i] = i;
```

```
int totalCost = 0;  
int count = 0;
```

```
printf("Edges in the constructed MST:\n");
```

```
for (int i = 0; i < E && count < V - 1; i++) {  
    int u = edges[i].u;  
    int v = edges[i].v;  
    int w = edges[i].w;  
  
    if (find(u) != find(v)) {  
        unionSet(u, v);
```

```

        printf("%d -- %d == %d\n", u, v, w);
        totalCost += w;
        count++;
    }
}

printf("Minimum Spanning Tree: %d\n", totalCost);

return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

You are designing a data compression tool that uses Huffman Coding to encode data efficiently.

Write a program to read a set of characters along with their corresponding Huffman variable-length binary codes, and use them to generate the encoded binary string for a given input message.

It is guaranteed that every character in the input message has a corresponding Huffman code in the provided code table.

Input Format

The first line contains an integer N (integer), representing the number of characters in the code table.

Each of the next N lines contains a character C and its corresponding Huffman code (a binary string of 0s and 1s), separated by a space.

The last line contains the input message (a string of characters) that needs to be encoded. Every character in the message is guaranteed to have a corresponding entry in the code table.

Output Format

The output prints 'Coded Data:' followed by a binary string (a sequence of 0s and 1s) representing the encoded message, obtained by concatenating the Huffman code of each character in the input message in order.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 10

D 0100

U 0000

F 0101

M 0001

A 0010

N 0011

C 1100

O 1101

E 1110

H 1111

HUFF

Output: Coded Data: 1111000001010101

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    int N;
```

```
    scanf("%d", &N);
```

```
    char ch[26];
```

```
    char code[26][25];
```

```
    // Read code table
```

```
    for (int i = 0; i < N; i++) {
```

```
        scanf(" %c %s", &ch[i], code[i]);
```

```
    }
```

```
    char msg[105];
```

```
    scanf("%s", msg);
```

```
    printf("Coded Data: ");
```



```
// Encode message
for (int i = 0; msg[i] != '\0'; i++) {
    for (int j = 0; j < N; j++) {
        if (msg[i] == ch[j]) {
            printf("%s", code[j]);
            break;
        }
    }
}

return 0;
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KARTHIK YARNAGULA

Email: vtu30340@veltech.edu.in

Roll no: vtu30340

Phone: null

Branch: VTU

Department: CSE with CS

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 7_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 13

Section 1 : MCQ

1. The frequencies of letters in a file are:

$s = 17, k = 24, y = 37$

A Huffman tree is constructed, and we assign 0 to the left branch and 1 to the right branch at each node.

Which of the following is the Huffman code for the letter 's'?

Answer

10

Status : Correct

Marks : 1/1

2. In Huffman coding, does data in a tree always occur?

Answer

Leaves

Status : Correct

Marks : 1/1

3. Which of the following algorithms is the best approach for solving Huffman codes?

Answer

Greedy algorithm

Status : Correct

Marks : 1/1

4. From the following given tree, what is the computed codeword for 'c'?

Answer

011

Status : Wrong

Marks : 0/1

5. From the following given tree, what is the code word for the character 'a'?

Answer

011

Status : Correct

Marks : 1/1

6. Kruskal's algorithm is used to _____.

Answer

Find minimum spanning tree

Status : Correct

Marks : 1/1

7. Consider the following graph:

Which edges would be included in the minimum spanning tree using Prim's algorithm starting from vertex A?

Answer

AB, BD, DE, EF, FC

Status : Correct

Marks : 1/1

8. Which of the following is true?

Answer

Prim's algorithm initializes with a vertex

Status : Correct

Marks : 1/1

9. Prim's algorithm can be efficiently implemented using _____ for graphs with greater density.

Answer

d-ary heap

Status : Correct

Marks : 1/1

10. Which of the following is false about Prim's algorithm?

Answer

It constructs MST by selecting edges in increasing order of their weights

Status : Correct

Marks : 1/1

11. What is the worst-case time complexity of Prim's algorithm if the adjacency matrix is used?

Answer

$O(V^2)$

Status : Correct

Marks : 1/1

12. Consider a graph $G = (V, E)$, where $V = \{v_1, v_2, \dots, v_{100}\}$, $E = \{(v_i, v_j) \mid 1 \leq i < j \leq 100\}$ and the weight of the edge (v_i, v_j) is $i - j$. The weight of the minimum spanning tree of G is _____.

Answer

99

Status : Correct

Marks : 1/1

13. An undirected graph $G(V, E)$ contains n ($n > 2$) nodes named $v_1, v_2, \dots, v_n$. Two nodes v_i, v_j are connected if and only if $0 < |i - j| \leq 2$. Each edge (v_i, v_j) is assigned a weight $i + j$. A sample graph with $n = 4$ is shown below. What will be the cost of the minimum spanning tree (MST) of such a graph with n nodes?

Answer

$n^2 - n + 1$

Status : Correct

Marks : 1/1

14. An undirected graph G has n nodes. Its adjacency matrix is given by an $n \times n$ square matrix whose (i) diagonal elements are 0's and (ii) non-diagonal elements are 1's. which one of the following is TRUE?

Answer

Graph G has multiple distinct MSTs, each of cost $n-1$

Status : Correct

Marks : 1/1

15. Consider a weighted complete graph G on the vertex set $\{v_1, v_2, \dots, v_n\}$ such that the weight of the edge (v_i, v_j) is $2|i-j|$. The weight of a minimum spanning tree of G is:

Answer

$n-1$

Status : Wrong

Marks : 0/1

Vel Tech Group of Institutions

Name: KARTHIK YARNAGULA

Email: vtu30340@veltech.edu.in

Roll no: vtu30340

Phone: null

Branch: VTU

Department: CSE with CS

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 6_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 44.5

Section 1 : Coding

1. Problem Statement

Virat is given a group of individuals, and he wants to identify if there is an influential individual within the group. An influential individual is defined as someone who is followed by more people than they follow.

He is provided with a list of follow-up relationships among the individuals in the form of a matrix. Your task is to help Virat determine if there is an influential individual and, if so, find their index within the group.

Write a program to solve this problem using Warshall's Algorithm.

Input Format

The first line of input consists of an integer N, representing the number of individuals in the group.

The following N lines will contain N space-separated integers, representing the following relationship matrix for the individuals.

Output Format

If an influential individual is found, print "Influential Individual: X," where X is the index (0-based) of the influential individual.

Otherwise, print "There is no influential individual in the group".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 1 1 0

0 0 1 0

0 0 0 0

0 0 1 0

Output: Influential Individual: 2

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int a[n][n];
```

```
    // Input matrix
```

```
    for(int i = 0; i < n; i++)
```

```
        for(int j = 0; j < n; j++)
```

```
            scanf("%d", &a[i][j]);
```

```
    // Warshall's Algorithm (transitive closure)
```

```
    for(int k = 0; k < n; k++)
```

```
        for(int i = 0; i < n; i++)
```

```
            for(int j = 0; j < n; j++)
```



```

        if(a[i][k] && a[k][j])
            a[i][j] = 1;

// Check influential individual
for(int i = 0; i < n; i++) {
    int follow = 0, followers = 0;

    for(int j = 0; j < n; j++) {
        if(a[i][j]) follow++; // i follows j
        if(a[j][i]) followers++; // j follows i
    }

    if(followers > follow) {
        printf("Influential Individual: %d", i);
        return 0;
    }
}

printf("There is no influential individual in the group");

return 0;
}

```

Status : Partially correct

Marks : 4.5/10

2. Problem statement

Sarah has been tasked with developing a program to determine the shortest paths among nodes in a directed network represented as a graph containing nodes and weighted edges.

Her goal is to implement the Floyd-Warshall algorithm to compute the shortest path matrix for this directed network.

Input Format

The first line of input consists of an integer n , representing the number of nodes in the network.

The next n lines contain a space-separated integer, representing the adjacency matrix 'graph' of the network. Each integer is the weight of the edge from node i

to node j.

Output Format

The output prints the shortest path matrix as 'n x n' integers, where each element `graph[i][j]` represents the shortest distance from node 'i' to node 'j'.

Print each row of the matrix on a new line, with elements separated by a space, and if no path exists, 999 will be displayed.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
0 3 999 4
8 0 2 999
5 999 0 1
2 999 999 0
Output: 0 3 5 4
5 0 2 3
3 6 0 1
2 5 7 0

Answer

```
// You are using GCC
#include <stdio.h>
```

```
#define INF 999
```

```
int main() {
    int n;
    scanf("%d", &n);
```

```
    int dist[n][n];
```

```
    // Input matrix
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            scanf("%d", &dist[i][j]);
        }
    }
```

```

    }

    // Floyd-Warshall
    for(int k = 0; k < n; k++) {
        for(int i = 0; i < n; i++) {
            for(int j = 0; j < n; j++) {
                if(dist[i][k] + dist[k][j] < dist[i][j])
                    dist[i][j] = dist[i][k] + dist[k][j];
            }
        }
    }

    // Output
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            printf("%d ", dist[i][j]);
        }
        printf("\n");
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

You are working as a backend engineer for a drone delivery logistics company that operates across multiple cities. Each city is represented as a node, and the drone flight paths between cities are represented as directed edges with weights indicating the time (in minutes) it takes for a drone to travel between them.

Due to varying air traffic, regulations, and geography, the direct travel time between cities may not always be the fastest. Sometimes, going through intermediate cities results in shorter delivery times.

Your task is to build a system that calculates the shortest possible delivery time between every pair of cities, considering all possible intermediate routes.

To efficiently compute this, you're required to implement the Floyd-Warshall Algorithm.

Input Format

The first line of input contains an integer N representing the total number of cities.

The next N lines contains N space-separated integers, representing the delivery time matrix:

time[i][j] is the time (in minutes) taken to fly directly from City i to City j.

A value of 0 means the same city (i.e., $i == j$).

A value of 999 indicates no direct route between City i and City j.

Output Format

The output prints the shortest distance between every pair of cities.

If there is no shortest path between two cities, then print INF.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
0 5 999 10
999 0 3 999
999 999 0 1
999 999 999 0

Output: 0 5 8 9
INF 0 3 4
INF INF 0 1
INF INF INF 0

Answer

```
// You are using GCC  
#include <stdio.h>
```

```
#define INF 999
```

```
int main() {  
    int n;  
    scanf("%d", &n);
```

```
  
    int dist[n][n];
```

```
  
    // Input matrix  
    for(int i = 0; i < n; i++) {  
        for(int j = 0; j < n; j++) {  
            scanf("%d", &dist[i][j]);  
        }  
    }
```

```
  
    // Floyd-Warshall  
    for(int k = 0; k < n; k++) {  
        for(int i = 0; i < n; i++) {  
            for(int j = 0; j < n; j++) {  
                if(dist[i][k] != INF && dist[k][j] != INF &&  
                    dist[i][k] + dist[k][j] < dist[i][j]) {  
                    dist[i][j] = dist[i][k] + dist[k][j];  
                }  
            }  
        }  
    }
```

```
  
    // Output  
    for(int i = 0; i < n; i++) {  
        for(int j = 0; j < n; j++) {  
            if(dist[i][j] == INF)  
                printf("INF ");  
            else  
                printf("%d ", dist[i][j]);  
        }  
        printf("\n");  
    }
```

```
  
    return 0;
```

```
}
```

Status : Correct

Marks : 10/10

4. Problem Statement

Given the weights and values of n items, put these items in a knapsack of capacity W to get the maximum total value in the knapsack. In other words, given two integer arrays, $val[0..n-1]$ and $wt[0..n-1]$ which represent values and weights associated with n items, respectively. Also given an integer W which represents knapsack capacity, find out the maximum value subset of $val[]$ such that the sum of the weights of this subset is smaller than or equal to W . You cannot break an item, either pick the complete item or don't pick it.

Solve this problem using dynamic programming.

Input Format

The first line of input represents the number of items.

The second line of input consists of the values of each item separated by spaces.

The third line of input consists of the weights of each item separated by spaces.

The fourth line of input consists of an integer, which represents the maximum capacity of the knapsack.

Output Format

The output prints the maximum value of items that can be held by the knapsack.

Sample Test Case

Input: 3
60 100 120
10 20 30
50
Output: 220

Answer

```
// You are using GCC
#include <stdio.h>
```

```

int main() {
    int n, W;
    scanf("%d", &n);

    int val[n], wt[n];

    for(int i = 0; i < n; i++)
        scanf("%d", &val[i]);

    for(int i = 0; i < n; i++)
        scanf("%d", &wt[i]);

    scanf("%d", &W);
    int dp[n+1][W+1];

    // DP calculation
    for(int i = 0; i <= n; i++) {
        for(int w = 0; w <= W; w++) {
            if(i == 0 || w == 0)
                dp[i][w] = 0;
            else if(wt[i-1] <= w) {
                int a = val[i-1] + dp[i-1][w - wt[i-1]];
                int b = dp[i-1][w];
                dp[i][w] = (a > b) ? a : b;
            } else {
                dp[i][w] = dp[i-1][w];
            }
        }
    }

    printf("%d", dp[n][W]);

    return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

Sarah loves challenges and has recently taken up a new programming problem. She is fascinated by the classic "0/1 Knapsack Problem" and wants to create a program that can efficiently solve it.

The 0/1 Knapsack Problem involves selecting a combination of items with given weights and values to maximize the total value, while not exceeding a given weight capacity.

Write a program that takes input for the 0/1 Knapsack Problem and outputs the maximum value that can be achieved, as well as the selected weights that contribute to this maximum value.

Note: The weights are printed in reverse order of the input, meaning the algorithm starts checking from the last item and prints items that are included first.

Input Format

The first line contains an integer n , representing the number of items available.

The second line contains n integers separated by space, representing the values of the items. The i th integer corresponds to the value of the i th item.

The third line contains n integers separated by space, representing the weights of the items. The i th integer corresponds to the weight of the i th item.

The fourth line contains an integer W , representing the maximum weight capacity of the knapsack.

Output Format

The first line output displays "Maximum value: " followed by an integer, representing the maximum value that can be achieved.

The second line output displays "Selected weights: " followed by space-separated integers, representing the selected weights contributing to the maximum value.

Refer to the sample outputs for the exact format.

Sample Test Case

Input: 3

60 100 120

10 20 30

50

Output: Maximum value: 220

Selected weights: 30 20

Answer

// You are using GCC

#include <stdio.h>

```
int main() {
```

```
    int n, W;
```

```
    scanf("%d", &n);
```

```
    int val[n], wt[n];
```

```
    for(int i = 0; i < n; i++)
```

```
        scanf("%d", &val[i]);
```

```
    for(int i = 0; i < n; i++)
```

```
        scanf("%d", &wt[i]);
```

```
    scanf("%d", &W);
```

```
    int dp[n+1][W+1];
```

```
    // Build DP table
```

```
    for(int i = 0; i <= n; i++) {
```

```
        for(int w = 0; w <= W; w++) {
```

```
            if(i == 0 || w == 0)
```

```
                dp[i][w] = 0;
```

```
            else if(wt[i-1] <= w) {
```

```
                int a = val[i-1] + dp[i-1][w - wt[i-1]];
                int b = dp[i-1][w];
```

```
                dp[i][w] = (a > b) ? a : b;
```

```
            } else {
```

```
                dp[i][w] = dp[i-1][w];
```

```
            }
```

```
        }
```

```
    }
```

```
}  
// Backtrack to find selected weights (reverse order)  
int w = W;  
printf("Maximum value: %d\n", dp[n][W]);  
printf("Selected weights: ");  
  
for(int i = n; i > 0; i--) {  
    if(dp[i][w] != dp[i-1][w]) {  
        printf("%d ", wt[i-1]);  
        w -= wt[i-1];  
    }  
}  
return 0;  
}
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KARTHIK YARNAGULA

Email: vtu30340@veltech.edu.in

Roll no: vtu30340

Phone: null

Branch: VTU

Department: CSE with CS

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 6_COD

Attempt : 1

Total Mark : 40

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Ankit wants to share the job among his employees so that maximum profit can be obtained. The condition is that no two jobs for a particular employee overlap.

Given N jobs, where every job is represented by the following three elements:

Start Time

Finish Time

Profit or Value Associated (≥ 0)

Find the maximum-profit subset of jobs such that no two jobs in the subset overlap.

Input Format

The first line of the input consists of the value of n.

The next n lines of the inputs consist of the start, end, and profit.

Output Format

The output prints the optimal profit.

Refer to the sample input and output for format specifications.

Sample Test Case

Input: 4

3 10 20

1 2 50

6 19 100

2 100 200

Output: The optimal profit is 250

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
struct Job {  
    int start, finish, profit;  
};
```

```
// Function to sort jobs by finish time
```

```
void sort(struct Job arr[], int n) {  
    for(int i = 0; i < n - 1; i++) {  
        for(int j = i + 1; j < n; j++) {  
            if(arr[i].finish > arr[j].finish) {  
                struct Job temp = arr[i];  
                arr[i] = arr[j];  
                arr[j] = temp;  
            }  
        }  
    }  
}
```

```

// Find last non-conflicting job
int latestNonConflict(struct Job arr[], int i) {
    for(int j = i - 1; j >= 0; j--) {
        if(arr[j].finish <= arr[i].start)
            return j;
    }
    return -1;
}

```

```

int maxProfit(struct Job arr[], int n) {
    int dp[n];
    dp[0] = arr[0].profit;

    for(int i = 1; i < n; i++) {
        int incl = arr[i].profit;
        int l = latestNonConflict(arr, i);
        if(l != -1)
            incl += dp[l];

        dp[i] = (incl > dp[i-1]) ? incl : dp[i-1];
    }

    return dp[n-1];
}

```

```

int main() {
    int n;
    scanf("%d", &n);

    struct Job arr[n];

    for(int i = 0; i < n; i++) {
        scanf("%d %d %d", &arr[i].start, &arr[i].finish, &arr[i].profit);
    }

    sort(arr, n);

    int result = maxProfit(arr, n);
    printf("The optimal profit is %d", result);

    return 0;
}

```

}

Status : Correct

Marks : 10/10

2. Problem Statement

In a city, there are several locations connected by roads. Each road has a certain distance. You are tasked with finding the shortest distances between all pairs of locations in the city using the Floyd-Warshall algorithm.

Write a program that calculates the shortest path between all pairs of locations in the city.

Example

Input:

4

4

0 1 3

1 2 1

2 3 1

0 2 5

Output:

0 3 4 5

3 0 1 2

4 1 0 1

5 2 1 0

Explanation:

The city has 4 locations (0 to 3).

There are 4 roads connecting the locations.

The roads and their distances are:

- 0 1 with distance 3
- 1 2 with distance 1
- 2 3 with distance 1
- 0 2 with distance 5

Initially, the distance matrix is set to infinity (INF) except for direct roads and diagonal values (0).

The Floyd-Warshall algorithm updates the matrix by checking intermediate paths.

The shortest path from 0 to 2 (via 1) is updated to 4 (0 1 2) instead of 5.

The shortest path from 0 to 3 is 5 (0 1 2 3).

The final distance matrix is printed, showing the shortest distances.

0 3 4 5

3 0 1 2

4 1 0 1

5 2 1 0

Input Format

The first line contains an integer N, representing the number of locations in the city.

The second line contains an integer M, representing the number of roads in the city.

The next M lines each contain three space-separated integers: u, v, w where:

- u and v represent two locations connected by a road.
- w represents the distance between u and v.

Output Format

The output prints a matrix, where the value at the i-th row and j-th column represents the shortest distance between location i and location j.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

4

0 1 3

1 2 1

2 3 1

0 2 5

Output: 0 3 4 5

3 0 1 2

4 1 0 1

5 2 1 0

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#define INF 99999
```

```
int main() {
```

```
    int n, m;
```

```
    scanf("%d", &n);
```

```
    scanf("%d", &m);
```

```
    int dist[n][n];
```

```
    // Initialize matrix
```

```
    for(int i = 0; i < n; i++) {
```

```
        for(int j = 0; j < n; j++) {
```

```
            if(i == j)
```

```
                dist[i][j] = 0;
```

```
            else
```

```
                dist[i][j] = INF;
```

```
        }
```

```
    }
```

```
    // Input edges
```

```
    for(int i = 0; i < m; i++) {
```



```

int u, v, w;
scanf("%d %d %d", &u, &v, &w);
dist[u][v] = w;
dist[v][u] = w; // undirected
}

// Floyd-Warshall
for(int k = 0; k < n; k++) {
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            if(dist[i][j] > dist[i][k] + dist[k][j])
                dist[i][j] = dist[i][k] + dist[k][j];
        }
    }
}

// Output
for(int i = 0; i < n; i++) {
    for(int j = 0; j < n; j++) {
        printf("%d ", dist[i][j]);
    }
    printf("\n");
}

return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Sharon is developing a program that analyzes the reachability of nodes in a graph. Given an adjacency matrix representing the graph and its size, her goal is to determine whether there exists a path from one vertex to another for all pairs of vertices.

Write a program to help Sharon perform this reachability analysis using Warshall's Algorithm.

Input Format

The first line of input consists of an integer N, representing the number of vertices in the graph.

The following N lines consist of N space-separated integers, forming the adjacency matrix graph, where graph[i][j] is either 0 or 1.

Output Format

The output prints an NxN matrix representing the reachability matrix for all pairs of vertices.

Each element in the matrix should be 1 if there is a path from the corresponding row vertex to the column vertex, and 0 otherwise.

Sample Test Case

Input: 4

0 1 0 0

0 0 1 0

0 0 0 1

0 0 0 0

Output: 0 1 1 1

0 0 1 1

0 0 0 1

0 0 0 0

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
int main() {
```

```
    int n;
```

```
    scanf("%d", &n);
```

```
    int a[n][n];
```

```
    // Input adjacency matrix
```

```
    for(int i = 0; i < n; i++) {
```

```
        for(int j = 0; j < n; j++) {
```

```
            scanf("%d", &a[i][j]);
```

```
        }
```

```
    }
```

```
    // Warshall's Algorithm
```

```

for(int k = 0; k < n; k++) {
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            if(a[i][j] == 0 && (a[i][k] && a[k][j]))
                a[i][j] = 1;
        }
    }
}

// Output reachability matrix
for(int i = 0; i < n; i++) {
    for(int j = 0; j < n; j++) {
        printf("%d ", a[i][j]);
    }
    printf("\n");
}

return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

You and your group of adventurous friends are planning a once-in-a-lifetime outdoor adventure trip to an exotic location. To ensure that your adventure goes smoothly, you need to carefully choose what to carry with you, as you have a limited carrying capacity. Your goal is to maximize the overall value of the items you bring while staying within your weight limit.

Write a program that helps you and your friends determine the optimal selection of items to bring on your adventure trip. The program should implement the 0–1 Knapsack Problem algorithm to find the best combination of items to maximize their total value while respecting the weight constraint.

Input Format

The first line contains an integer n , representing the number of items available for selection.

The second line contains an integer limit, representing the maximum weight limit your group can carry in a bag.

The next n lines contain space-separated information for each item:

- The name of the item is a string (without spaces).
- The weight of the item.
- The value of the item.

Output Format

The first line of output displays "Total value: " followed by an integer representing the total value of the selected items.

The second line of output displays "Total weight: " followed by an integer representing the total weight of the selected items.

Refer to the sample outputs for the exact format.

Sample Test Case

Input: 2

25

Bag 10 250

Note 15 430

Output: Total value: 680

Total weight: 25

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    int n, W;
```

```
    scanf("%d", &n);
```

```
    scanf("%d", &W);
```

```
    char name[n][50];
```

```
    int wt[n], val[n];
```

```

for(int i = 0; i < n; i++) {
    scanf("%s %d %d", name[i], &wt[i], &val[i]);
}

int dp[n+1][W+1];

// Initialize DP
for(int i = 0; i <= n; i++) {
    for(int w = 0; w <= W; w++) {
        if(i == 0 || w == 0)
            dp[i][w] = 0;
        else if(wt[i-1] <= w) {
            int include = val[i-1] + dp[i-1][w - wt[i-1]];
            int exclude = dp[i-1][w];
            dp[i][w] = (include > exclude) ? include : exclude;
        } else {
            dp[i][w] = dp[i-1][w];
        }
    }
}

// Backtrack to find total weight
int res = dp[n][W];
int w = W, totalWeight = 0;

for(int i = n; i > 0 && res > 0; i--) {
    if(res != dp[i-1][w]) {
        totalWeight += wt[i-1];
        res -= val[i-1];
        w -= wt[i-1];
    }
}

printf("Total value: %d\n", dp[n][W]);
printf("Total weight: %d", totalWeight);

return 0;
}

```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KARTHIK YARNAGULA

Email: vtu30340@veltech.edu.in

Roll no: vtu30340

Phone: null

Branch: VTU

Department: CSE with CS

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 6_MCQ

Attempt : 1

Total Mark : 15

Marks Obtained : 14

Section 1 : MCQ

1. Which of the following is/are property/properties of a dynamic programming problem?

Answer

Both optimal substructure and overlapping subproblems

Status : Correct

Marks : 1/1

2. If an optimal solution can be created for a problem by constructing optimal solutions for its subproblems, the problem possesses _____ property.

Answer

Optimal substructure

Status : Correct

Marks : 1/1

3. In dynamic programming, the technique of storing the previously calculated values is called _____.

Answer

Memoization

Status : Correct

Marks : 1/1

4. The following paradigm can be used to find the solution to the problem in minimum time:

Given a set of non-negative integers, and a value K, determine if there is a subset of the given set with a sum equal to K.

Answer

Dynamic Programming

Status : Correct

Marks : 1/1

5. Which of the following is an example of a problem that can be solved using memoization in dynamic programming?

Answer

Computing the edit distance between two strings

Status : Correct

Marks : 1/1

6. The Floyd-Warshall algorithm for all-pairs shortest paths computation is based on

Answer

Dynamic Programming paradigm

Status : Correct

Marks : 1/1

7. Floyd-Warshall algorithm utilizes _____ to solve the all-pairs shortest paths problem on a directed graph in _____ time.

Answer

Dynamic programming, $\theta(V^3)$

Status : Correct

Marks : 1/1

8. Floyd Warshall's Algorithm can be applied to _____.

Answer

Directed graphs

Status : Correct

Marks : 1/1

9. In the Floyd Warshall algorithm, how many iterations are required to find the shortest path between all pairs of vertices in a graph with N vertices?

Answer

N

Status : Correct

Marks : 1/1

10. Consider a directed weighted graph with V vertices and E edges, where each edge weight is represented as a 32-bit signed integer. What will be the maximum possible value for the edge weight to guarantee the correct shortest path calculation using the Floyd Warshall algorithm?

Answer

$2^{15} - 1$

Status : Wrong

Marks : 0/1

11. 0/1 knapsack is based on _____ method.

Answer

Dynamic programming

Status : Correct

Marks : 1/1

12. Fractional knapsack problem differs from 0/1 knapsack because:

Answer

Items can be divided into fractions

Status : Correct

Marks : 1/1

13. Which of the following methods can be used to solve the Knapsack problem?

Answer

All the mentioned options

Status : Correct

Marks : 1/1

14. What is the primary advantage of using the Branch and Bound approach for the Knapsack problem?

Answer

It systematically explores possible solutions while pruning non-promising ones.

Status : Correct

Marks : 1/1

15. Which of the following problems cannot be solved optimally by greedy approach?

Answer

0/1 Knapsack

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: KARTHIK YARNAGULA

Email: vtu30340@veltech.edu.in

Roll no: vtu30340

Phone: null

Branch: VTU

Department: CSE with CS

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 5_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Given an array of integers, count the number of smaller elements that appear to the right of each element. Implement a solution using merge sort to count these elements efficiently. Output the result as an array of counts corresponding to each element in the original array.

Example

Input:

4

5 2 6 1

Output:

2 1 1 0

Explanation:

To the right of 5, there are 2 smaller elements (2 and 1).

To the right of 2, there is only 1 smaller element (1).

To the right of 6, there is 1 smaller element (1).

To the right of 1, there is 0 smaller element.

Input Format

The first line of input contains a single integer n, representing the length of the array.

The second line contains n space-separated integers, representing the elements of the array.

Output Format

The output prints a single line containing the counts of smaller elements for each value in the array, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

5 2 6 1

Output: 2 1 1 0

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#define MAX 20
```

```
typedef struct {
```

```
    int val;
```

```
    int idx;
```

```
} Element;
```

```
void merge(Element arr[], int left, int mid, int right, int count[]) {  
    int n1 = mid - left + 1;  
    int n2 = right - mid;
```

```
    Element L[n1], R[n2];
```

```
    for(int i = 0; i < n1; i++) L[i] = arr[left + i];  
    for(int i = 0; i < n2; i++) R[i] = arr[mid + 1 + i];
```

```
    int i = 0, j = 0, k = left;  
    int rightCount = 0;
```

```
    while(i < n1 && j < n2) {  
        if(L[i].val > R[j].val) {  
            // L[i] has one more smaller element to its right  
            count[L[i].idx] += n2 - j;  
            arr[k++] = L[i++];  
        } else {  
            arr[k++] = R[j++];  
        }  
    }  
    while(i < n1) arr[k++] = L[i++];  
    while(j < n2) arr[k++] = R[j++];  
}
```

```
void mergeSort(Element arr[], int left, int right, int count[]) {  
    if(left < right) {  
        int mid = left + (right - left) / 2;  
        mergeSort(arr, left, mid, count);  
        mergeSort(arr, mid + 1, right, count);  
        merge(arr, left, mid, right, count);  
    }  
}
```

```
int main() {  
    int n;  
    scanf("%d", &n);  
    int nums[MAX];  
    for(int i = 0; i < n; i++) scanf("%d", &nums[i]);  
  
    Element arr[MAX];  
    int count[MAX] = {0};
```

```

for(int i = 0; i < n; i++) {
    arr[i].val = nums[i];
    arr[i].idx = i;
}

mergeSort(arr, 0, n - 1, count);

for(int i = 0; i < n; i++)
    printf("%d ", count[i]);
printf("\n");

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Ravi, a mathematics teacher, is helping his students understand quadratic equations. He wants to apply the quadratic formula $[A * x^2 + B * x + C]$ to each number in a sorted array. Once the formula is applied, the modified numbers need to be sorted again. Ravi plans to use QuickSort for efficient sorting. Help Ravi by implementing a program that performs these operations.

Input Format

The first line contains an integer n , representing the size of the array.

The second line contains n space-separated integers, representing the elements of the array in ascending order.

The third line contains three integers A , B , and C , the coefficients of the quadratic formula.

Output Format

The first line of output prints: Array after applying the equation and sorting

The subsequent lines print: <sorted_value>

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

-1 0 1 2 3 4

-1 2 -1

Output: Array after applying the equation and sorting:

-9 -4 -4 -1 -1 0

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
// QuickSort partition function
```

```
int partition(int arr[], int low, int high) {
```

```
    int pivot = arr[high];
```

```
    int i = low - 1;
```

```
    for(int j = low; j <= high - 1; j++) {
```

```
        if(arr[j] <= pivot) {
```

```
            i++;
```

```
            int temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;
```

```
        }
```

```
    }
```

```
    int temp = arr[i+1]; arr[i+1] = arr[high]; arr[high] = temp;
```

```
    return i + 1;
```

```
}
```

```
// QuickSort function
```

```
void quickSort(int arr[], int low, int high) {
```

```
    if(low < high) {
```

```
        int pi = partition(arr, low, high);
```

```
        quickSort(arr, low, pi - 1);
```

```
        quickSort(arr, pi + 1, high);
```

```
    }
```

```
}
```

```
// Quadratic function
```

```
int quadratic(int x, int A, int B, int C) {
```

```
    return A*x*x + B*x + C;
```

```
}
```

```

int main() {
    int n;
    scanf("%d", &n);

    int arr[100];
    for(int i = 0; i < n; i++) scanf("%d", &arr[i]);

    int A, B, C;
    scanf("%d %d %d", &A, &B, &C);

    // Apply quadratic formula to each element
    for(int i = 0; i < n; i++) {
        arr[i] = quadratic(arr[i], A, B, C);
    }

    // Sort the array using QuickSort
    quickSort(arr, 0, n - 1);

    printf("Array after applying the equation and sorting:\n");
    for(int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Amit, an e-commerce manager, is working on an inventory management system to track the top-selling products. Given the sales data of various products, he wants to quickly determine the Kth best-selling product to make informed restocking decisions.

Help Amit implement Heap Sort to efficiently find the Kth best-selling product from the given sales data.

Input Format

The first line of input consists of an integer n , representing the number of products.

The second line consists of n space-separated integers, representing the sales data of each product.

The third line contains an integer K , representing the rank of the product to retrieve.

Output Format

The output prints the K th best-selling product based on the given sales data.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
10 20 15 5 25
2

Output: 20

Answer

```
#include <stdio.h>

void heapify(int arr[], int n, int i) {
    int largest = i;    // Initialize largest as root
    int left = 2 * i + 1; // left child
    int right = 2 * i + 2; // right child

    if(left < n && arr[left] > arr[largest])
        largest = left;
    if(right < n && arr[right] > arr[largest])
        largest = right;

    if(largest != i) {
        int temp = arr[i]; arr[i] = arr[largest]; arr[largest] = temp;
        heapify(arr, n, largest);
    }
}
```



```

}

void heapSort(int arr[], int n) {
    // Build max heap
    for(int i = n/2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // Extract elements one by one
    for(int i = n - 1; i > 0; i--) {
        int temp = arr[0]; arr[0] = arr[i]; arr[i] = temp;
        heapify(arr, i, 0);
    }
}

int main() {
    int n;
    scanf("%d", &n);
    int arr[n];
    for(int i = 0; i < n; i++) scanf("%d", &arr[i]);

    int K;
    scanf("%d", &K);

    heapSort(arr, n);

    // After heapSort, array is sorted in ascending order
    // Kth best-selling = n-K th index
    printf("%d\n", arr[n-K]);

    return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Tom is working in a warehouse inventory system, where the item IDs are assigned sequentially in ascending order.

He wants to develop a program using recursive binary search to efficiently

determine the closest item ID that is less than or equal to a given target ID.

The program takes the total number of items and their sorted IDs as input, assisting warehouse staff in quickly identifying the closest available item less than or equal to the target ID. Help Tom in this program.

Input Format

The first line of input consists of an integer N, representing the number of items in the warehouse.

The second line consists of N space-separated integers, representing the sorted list of item IDs.

The third line consists of an integer representing the target item ID.

Output Format

The output prints "The closest item ID less than or equal to X is Y", where X is the target ID and Y is the closest item ID less than or equal to X.

If no such element exists, print "No closest item with an ID less than or equal to X exists in the warehouse", where X is the target ID.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7

17 25 38 47 51 62 79

50

Output: The closest item ID less than or equal to 50 is 47

Answer

You are using Python

```
def closest_binary_search(arr, target, low, high, closest):
```

```
    if low > high:
```

```
        return closest
```

```
    mid = (low + high) // 2
```

```

if arr[mid] == target:
    return arr[mid] # Exact match found
elif arr[mid] < target:
    # arr[mid] is a candidate for closest <= target
    return closest_binary_search(arr, target, mid + 1, high, arr[mid])
else:
    # arr[mid] > target, ignore right half
    return closest_binary_search(arr, target, low, mid - 1, closest)

# Read input
N = int(input())
item_ids = list(map(int, input().split()))
target_id = int(input())

# Perform search
result = closest_binary_search(item_ids, target_id, 0, N - 1, None)

# Print output
if result is None:
    print(f"No closest item with an ID less than or equal to {target_id} exists in the warehouse")
else:
    print(f"The closest item ID less than or equal to {target_id} is {result}")

```

Status : Correct

Marks : 10/10

5. Problem Statement

Rahul owns a bookstore and maintains a sorted list of ISBN numbers of books in his inventory. When a customer asks for a specific book, Rahul wants to quickly check whether the book is available in his inventory.

Write a program that implements a non-recursive binary search to determine if a given ISBN exists in the sorted list of ISBN numbers.

Input Format

The first line contains an integer N, representing the number of ISBN numbers in the inventory.

The second line contains N space-separated integers representing the sorted list

of ISBN numbers.

The third line contains a single integer X, the ISBN number to be searched.

Output Format

If the ISBN X is found in the inventory, print: "Book with ISBN X is available at index Y" where Y is the zero-based index of the ISBN in the list.

If the ISBN X is not found, print: "Book with ISBN X is not available in the inventory"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

2 3 40 50 20

3

Output: Book with ISBN 3 is available at index 1

Answer

You are using Python

Non-recursive binary search function

```
def binary_search(arr, target):
```

```
    low = 0
```

```
    high = len(arr) - 1
```

```
    while low <= high:
```

```
        mid = (low + high) // 2
```

```
        if arr[mid] == target:
```

```
            return mid # Target found
```

```
        elif arr[mid] < target:
```

```
            low = mid + 1
```

```
        else:
```

```
            high = mid - 1
```

```
    return -1 # Target not found
```

Read input

```
N = int(input())
```

```
isbn_list = list(map(int, input().split()))
```

```
X = int(input())
```

```
# Perform search
```

```
index = binary_search(isbn_list, X)
```

```
# Print output
```

```
if index != -1:
```

```
    print(f"Book with ISBN {X} is available at index {index}")
```

```
else:
```

```
    print(f"Book with ISBN {X} is not available in the inventory")
```

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KARTHIK YARNAGULA

Email: vtu30340@veltech.edu.in

Roll no: vtu30340

Phone: null

Branch: VTU

Department: CSE with CS

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 5_COD

Attempt : 1

Total Mark : 40

Marks Obtained : 30

Section 1 : Coding

1. Problem Statement

Sam needs to sort an array of integers using the divide-and-conquer method. He wants to implement the merge sort algorithm, displaying the array after each iteration to track the sorting process.

Help him write a program that divides the array, merges it back, and prints the array.

Input Format

The first line of input consists of an integer n , denoting the array size.

The second line consists of n space-separated integers, representing the array of elements.

Output Format

The first line of output prints "Given Array" followed by a (String), indicating the start of the initial array display.

The second line of output prints the array elements (integers) separated by spaces.

After each merge iteration, the output prints "After iteration X", where X is the iteration number (starting from 1). This is followed by the (String) "After iteration X" to indicate the iteration count.

On the next line, the current state of the array after that iteration is printed, showing the array elements (integers) separated by spaces.

Finally, after all iterations are completed, the output prints "Sorted Array" followed by a (String), indicating the sorted array.

The final line prints the sorted array (integers), with each element separated by spaces.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6

4 1 5 3 2 6

Output: Given Array

4 1 5 3 2 6

After iteration 1

1 4 5 3 2 6

After iteration 2

1 4 5 3 2 6

After iteration 3

1 4 5 2 3 6

After iteration 4

1 4 5 2 3 6
After iteration 5
1 2 3 4 5 6
Sorted Array
1 2 3 4 5 6

Answer

```
#include <stdio.h>
```

```
int max(int a, int b) {  
    return a > b ? a : b;  
}
```

```
int main() {  
    int n;  
    scanf("%d", &n);
```

```
    int weights[10], values[10];  
    for(int i = 0; i < n; i++) scanf("%d", &weights[i]);  
    for(int i = 0; i < n; i++) scanf("%d", &values[i]);
```

```
    int C;  
    scanf("%d", &C);
```

```
    // Calculate sum and average of values
```

```
    int sum = 0;  
    for(int i = 0; i < n; i++) sum += values[i];  
    double avg = (double)sum / n;
```

```
    // 0/1 Knapsack using dynamic programming
```

```
    int dp[31] = {0}; // dp[w] = max value for weight w  
    for(int i = 0; i < n; i++) {  
        for(int w = C; w >= weights[i]; w--) {  
            dp[w] = max(dp[w], dp[w - weights[i]] + values[i]);  
        }  
    }
```

```
    // Print results
```

```
    printf("Sum of values: %d\n", sum);  
    printf("Average of values: %.2lf\n", avg);  
    printf("Maximum amount: %d\n", dp[C]);
```



```
    return 0;  
}
```

Status : Wrong

Marks : 0/10

2. Problem Statement

Micheal is tasked with developing a system to manage and sort student records for a university. Each student record contains the student's name, GPA, age, and major. The goal is to implement a program that allows the user to input multiple student records and then sort these records based on a chosen criterion: GPA, age, or major.

Can you guide Micheal in developing the program using a quick sort algorithm?

Input Format

The first line of input consists of an integer n , representing the number of student records.

For each student record, the following lines consist of:

- Name (string)
- GPA (float)
- Age (integer)
- Major (string)

The last line of input consists of the sorting criterion, which can be one of the following strings:

- "gpa"
- "age"
- "major"

Output Format

The first line of output will print:

Sorted Student Records:

The output should display a sorted list of student records in a tabular format

based on the chosen criterion. The GPA value should be rounded off to two decimal places.

Table Header

"Name\t\tGPA\t\tAge\t\tMajor"

Table Content

For each student, the format of the output will be:

"Name\tGPA\tAge\t\tMajor"

- The values should be aligned as follows:
- Name: Left-aligned, up to 10 characters.
- GPA: Displayed with two decimal places.
- Age: Left-aligned, up to 3 digits.
- Major: Left-aligned, up to 15 characters.

Note: "\t" - It creates a horizontal tab space

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4

Harish

7.8

23

ece

Trell

8

21

cse

Kamalesh

7

25

it
Maaran
5.6
20
civil
age

Output: Sorted Student Records:

Name	GPA	Age	Major
Maaran	5.60	20	civil
Trell	8.00	21	cse
Harish	7.80	23	ece
Kamalesh	7.00	25	it

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>
```

```
#define MAX 10
```

```
typedef struct {
    char name[20];
    float gpa;
    int age;
    char major[20];
} Student;
```

```
// Swap two student records
void swap(Student *a, Student *b) {
    Student temp = *a;
    *a = *b;
    *b = temp;
}
```

```
// Partition function for Quick Sort
int partition(Student arr[], int low, int high, char criterion[]) {
    Student pivot = arr[high];
    int i = low - 1;
```

```
    for(int j = low; j < high; j++) {
        int cmp = 0;
        if(strcmp(criterion, "gpa") == 0) {
```

```

        if(arr[j].gpa < pivot.gpa) cmp = 1;
    } else if(strcmp(criterion, "age") == 0) {
        if(arr[j].age < pivot.age) cmp = 1;
    } else if(strcmp(criterion, "major") == 0) {
        if(strcmp(arr[j].major, pivot.major) < 0) cmp = 1;
    }
    if(cmp) {
        i++;
        swap(&arr[i], &arr[j]);
    }
}
swap(&arr[i + 1], &arr[high]);
return i + 1;
}

```

// Quick Sort function

```

void quickSort(Student arr[], int low, int high, char criterion[]) {
    if(low < high) {
        int pi = partition(arr, low, high, criterion);
        quickSort(arr, low, pi - 1, criterion);
        quickSort(arr, pi + 1, high, criterion);
    }
}

```

```

int main() {
    int n;
    scanf("%d", &n);
    Student students[MAX];
    for(int i = 0; i < n; i++) {
        scanf("%s", students[i].name);
        scanf("%f", &students[i].gpa);
        scanf("%d", &students[i].age);
        scanf("%s", students[i].major);
    }
}

```

```

char criterion[10];
scanf("%s", criterion);

```

```

quickSort(students, 0, n - 1, criterion);

```

```

printf("Sorted Student Records:\n");
printf("Name\t\tGPA\t\tAge\t\tMajor\n");

```

```
for(int i = 0; i < n; i++) {  
    printf("%-10s\t%.2f\t%-3d\t%-15s\n", students[i].name, students[i].gpa,  
students[i].age, students[i].major);  
}  
  
return 0;  
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Ragavi, a computer scientist, is working on efficient text processing for search engines. She has a list of words in random order and needs to sort them in lexicographic order using Heap Sort to improve search performance.

Help Ragavi implement Heap Sort to arrange the words in lexicographic order efficiently.

Input Format

The first line consists of an integer N denoting the number of strings.

The next line consists of N space-separated strings.

Output Format

The output displays N space-separated sorted strings in lexicographic order.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 10

banana pineapple mango chikoo jack star guava lemon tomato orange

Output: banana chikoo guava jack lemon mango orange pineapple star tomato

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>
```

```
#define MAX 20
```

```
// Swap two strings
void swap(char a[], char b[]) {
    char temp[50];
    strcpy(temp, a);
    strcpy(a, b);
    strcpy(b, temp);
}
```

```
// Heapify function for max heap
void heapify(char arr[][50], int n, int i) {
    int largest = i; // Initialize largest as root
    int left = 2*i + 1;
    int right = 2*i + 2;
```

```
    // Compare left child
    if(left < n && strcmp(arr[left], arr[largest]) > 0)
        largest = left;
    // Compare right child
    if(right < n && strcmp(arr[right], arr[largest]) > 0)
        largest = right;
```

```
    // Swap and continue heapifying if root is not largest
    if(largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}
```

```
// Heap sort function
void heapSort(char arr[][50], int n) {
    // Build max heap
    for(int i = n/2 - 1; i >= 0; i--)
        heapify(arr, n, i);
```

```
    // Extract elements from heap
    for(int i = n-1; i >= 0; i--) {
```

```

        swap(arr[0], arr[i]); // Move current root to end
        heapify(arr, i, 0); // Call heapify on reduced heap
    }
}

```

```

int main() {
    int N;
    scanf("%d", &N);
    char words[MAX][50];

    for(int i = 0; i < N; i++)
        scanf("%s", words[i]);

    heapSort(words, N);

    for(int i = 0; i < N; i++)
        printf("%s ", words[i]);

    printf("\n");
    return 0;
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Imagine you are working as a librarian in a large library. The library uses an automated system to keep track of book positions on the shelves. Each book is assigned a unique ID, but some IDs are more common than others due to the popularity of certain editions.

The system logs the positions of books in a sorted list based on their IDs. You need to find the first and last positions of a specific book ID in this sorted list using recursive binary search.

Example

Input:

10

1 2 2 2 2 3 4 8 8 8
8

Output:

7 9

Explanation:

The first and last positions of book ID 8 are [7, 9] since the index is 0-based.

Input Format

The first line of input consists of an integer n , representing the total number of book IDs.

The second line consists of a sorted list of n integers representing the book IDs.

The third line consists of an integer x , representing the specific book ID to find.

Output Format

The output displays indexes of the first and last occurrence of x , as space-separated integers.

If book ID x is not present in the shelves, print "NO OCCURRENCES".

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 10

1 2 2 2 2 3 4 8 8 8

8

Output: 7 9

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
int firstOccurrence(int arr[], int low, int high, int x) {
```



```

    if (high >= low) {
        int mid = low + (high - low) / 2;
        if ((mid == 0 || arr[mid - 1] < x) && arr[mid] == x)
            return mid;
        else if (arr[mid] < x)
            return firstOccurrence(arr, mid + 1, high, x);
        else
            return firstOccurrence(arr, low, mid - 1, x);
    }
    return -1;
}

```

```

int lastOccurrence(int arr[], int low, int high, int x, int n) {
    if (high >= low) {
        int mid = low + (high - low) / 2;
        if ((mid == n - 1 || arr[mid + 1] > x) && arr[mid] == x)
            return mid;
        else if (arr[mid] > x)
            return lastOccurrence(arr, low, mid - 1, x, n);
        else
            return lastOccurrence(arr, mid + 1, high, x, n);
    }
    return -1;
}

```

```

int main() {
    int n;
    scanf("%d", &n);
    int arr[20];
    for(int i = 0; i < n; i++) scanf("%d", &arr[i]);
    int x;
    scanf("%d", &x);

    int first = firstOccurrence(arr, 0, n - 1, x);
    if(first == -1) {
        printf("NO OCCURRENCES\n");
    } else {
        int last = lastOccurrence(arr, 0, n - 1, x, n);
        printf("%d %d\n", first, last);
    }

    return 0;
}

```

}

Status : Correct

Marks : 10/10

Vel Tech Group of Institutions

Name: KARTHIK YARNAGULA
Email: vtu30340@veltech.edu.in
Roll no: vtu30340
Phone: null
Branch: VTU
Department: CSE with CS
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 5_MCQ

Attempt : 1
Total Mark : 15
Marks Obtained : 14

Section 1 : MCQ

1. In heap sort, when does the actual sorting take place?

Answer

During the swap and heapify phase

Status : Correct

Marks : 1/1

2. In heap sort, which of the following heap operations is performed to build a valid heap from an unsorted array?

Answer

Heapify

Status : Correct

Marks : 1/1

3. In heap sort, the element that is swapped with the final element of the heap is _____, in the case of a max-heap, sorted in descending order.

Answer

Largest element

Status : Correct

Marks : 1/1

4. What is the worst-case time complexity of the heap sort algorithm?

Answer

$O(n \log n)$

Status : Correct

Marks : 1/1

5. In a quick sort algorithm, where are the smaller elements placed to the pivot during the partition process, assuming we are sorting in increasing order?

Answer

To the left of the pivot

Status : Correct

Marks : 1/1

6. Which of the following is true about Quicksort?

Answer

It is an in-place sorting algorithm

Status : Correct

Marks : 1/1

7. What is the worst-case time complexity of the Quicksort algorithm?

Answer

$O(n^2)$

Status : Correct

Marks : 1/1

8. Given a sorted array of 1000 unique integers, how many maximum comparisons does Binary Search make in the worst case?

Answer

10

Status : Correct

Marks : 1/1

9. Suppose you are using Binary Search on a sorted array of even length (e.g., 8 elements). At each step, you calculate the middle as:

$\text{mid} = (\text{low} + \text{high}) // 2$

Which of the following statements is always true?

Answer

The mid index may not divide the array into equal halves in all steps.

Status : Correct

Marks : 1/1

10. You apply Binary Search on the sorted array [3, 7, 10, 15, 21, 30] to find the element 13. Which sequence of middle elements will Binary Search compare with before declaring that the element is not present?

Answer

10 15 21

Status : Wrong

Marks : 0/1

11. You are using Binary Search to find the first occurrence of a value x in a sorted array that may contain duplicates. Which of the following modifications is necessary?

Answer

When $x == \text{arr}[\text{mid}]$, move high to mid - 1.

Status : Correct

Marks : 1/1

12. A student implements Binary Search and calculates the middle index as:

$\text{mid} = (\text{low} + \text{high}) // 2$

But when the input values of low and high are both large (e.g., low = 2_000_000_000, high = 2_000_000_010), the program crashes due to integer overflow. What is the best fix?

Answer

Use $\text{mid} = \text{low} + (\text{high} - \text{low}) // 2$

Status : Correct

Marks : 1/1

13. Is Merge Sort a stable sorting algorithm?

Answer

Yes, always stable.

Status : Correct

Marks : 1/1

14. Merge sort is _____.

Answer

Comparison-based sorting algorithm

Status : Correct

Marks : 1/1

15. Which of the following methods is used for sorting in merge sort?

Answer

merging

Status : Correct

Marks : 1/1

Vel Tech Group of Institutions

Name: KARTHIK YARNAGULA

Email: vtu30340@veltech.edu.in

Roll no: vtu30340

Phone: null

Branch: VTU

Department: CSE with CS

Batch: 2028

Degree: M.Tech - Computer Science Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS9L8

VTU_DAA_Week 4_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Meera is preparing for a science exhibition and needs to select a group of items whose total weight is exactly equal to a required target value. Each item can be selected at most once.

Given a list of item weights and a target weight K , determine whether it is possible to select a subset of items such that their sum is exactly K .

If such a subset exists, print any one valid subset. If no such subset exists, print -1.

This problem is derived from the Subset Sum problem, which is NP-Complete, and the solution must be implemented using Dynamic Programming.

Input Format

The first line contains two integers n and K

The second line contains n space-separated integers

Output Format

Print elements of one valid subset whose sum equals K

If no such subset exists, print -1

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5 9

3 34 4 12 5

Output: 5 4

Answer

```
// You are using GCC
```

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
int main() {  
    int n, K;  
    scanf("%d %d", &n, &K);  
  
    int arr[101];  
    for(int i = 0; i < n; i++) {  
        scanf("%d", &arr[i]);  
    }  
}
```

```
// DP table: dp[i][j] = true if sum j can be formed using first i items  
bool dp[101][1001] = {false};
```

```
for(int i = 0; i <= n; i++)  
    dp[i][0] = true; // sum 0 is always possible with 0 items
```

```
for(int i = 1; i <= n; i++) {  
    for(int j = 1; j <= K; j++) {  
        if(j >= arr[i-1])
```



```

        dp[i][j] = dp[i-1][j] || dp[i-1][j - arr[i-1]];
    else
        dp[i][j] = dp[i-1][j];
    }
}

if(!dp[n][K]) {
    printf("-1\n");
    return 0;
}

// Recover one valid subset
int sum = K;
int subset[101], idx = 0;
int i = n;
while(i > 0 && sum > 0) {
    if(dp[i-1][sum]) {
        // Current item not included
        i--;
    } else {
        // Current item included
        subset[idx++] = arr[i-1];
        sum -= arr[i-1];
        i--;
    }
}

// Print the subset
for(int j = 0; j < idx; j++) {
    printf("%d ", subset[j]);
}
printf("\n");

return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Rohit is a travel planner who must organize a business trip covering

multiple cities.

The travel cost between each pair of cities is given in a matrix.

He must:

Start from city 0
Visit every city exactly once
Return back to city 0
Your task is to compute:

The minimum possible travel cost
The sequence of cities visited
The average cost per step (excluding the final return step)

This problem is a classic Travelling Salesman Problem (TSP) solved using Dynamic Programming with Bitmasking (Held–Karp Algorithm).

Input Format

- The first line contains an integer n , the number of cities.
- The next n lines contain n space-separated integers, representing the cost matrix.

$\text{cost}[i][j]$ is the cost of traveling from city i to city j .

Output Format

Print the path:

The Path is: $0 \rightarrow \dots \rightarrow 0$

Print the minimum cost:

Minimum cost is X

Print the average cost per step:

Average cost per step is Y

Rounded to 2 decimal places.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Output: The Path is: 0--->2--->3--->1--->0

Minimum cost is 80

Average cost per step is 26.67

Answer

```
#include <stdio.h>
```

```
#include <limits.h>
```

```
int n, cost[10][10], dp[1<<10][10], parent[1<<10][10];
```

```
int main() {  
    scanf("%d",&n);  
    for(int i=0;i<n;i++)  
        for(int j=0;j<n;j++)  
            scanf("%d",&cost[i][j]);
```

```
    int FULL = 1<<n;  
    for(int mask=0;mask<FULL;mask++)  
        for(int i=0;i<n;i++)  
            dp[mask][i]=INT_MAX/2;
```

```
    dp[1][0]=0;
```

```
    for(int mask=1;mask<FULL;mask++)  
        for(int u=0;u<n;u++){  
            if(!(mask&(1<<u))) continue;  
            for(int v=0;v<n;v++){
```

```

        if(mask&(1<<v)) continue;
        int next=mask|(1<<v);
        if(dp[next][v]>dp[mask][u]+cost[u][v]){
            dp[next][v]=dp[mask][u]+cost[u][v];
            parent[next][v]=u;
        }
    }
}

int last=-1,minCost=INT_MAX, mask=FULL-1;
for(int i=1;i<=n;i++){
    if(dp[FULL-1][i]+cost[i][0]<minCost){
        minCost=dp[FULL-1][i]+cost[i][0];
        last=i;
    }
}

int path[11]; path[0]=0;
for(int i=n-1;i>0;i--){
    path[i]=last;
    int temp=parent[mask][last];
    mask^=1<<last;
    last=temp;
}
path[n]=0;

printf("The Path is: ");
for(int i=0;i<=n;i++){
    printf("%d",path[i]);
    if(i<n) printf("--->");
}
printf("\nMinimum cost is %d\n",minCost);
printf("Average cost per step is %.2lf\n",(double)minCost/n);

return 0;
}

```

Status : Wrong

Marks : 0/10

3. Problem Statement

Captain Aidan, a daring pirate, has discovered an ancient treasure cave

filled with valuable relics. However, the cave is protected by an ancient spell that limits the weight of the treasure he can carry. Aidan has a magic knapsack with a maximum weight capacity of W . Inside the cave, he finds n different treasures, each with a specific weight and value.

Since he can only take each treasure once, Aidan must carefully choose which items to take to maximize the total value without exceeding the weight limit of his knapsack.

Help Captain Aidan pick the most valuable treasures while staying within the weight limit using branch and bound technique

Input Format

The first line contains two integers n (the number of items) and W (the capacity of the knapsack), separated by a space.

The second line contains n integers, where the i -th integer represents the weight of the i -th item.

The third line contains n integers, where the i -th integer represents the value of the i -th item

Output Format

The output prints the single integer representing the maximum value obtained by filling the knapsack with the given set of items.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5 60
10 20 30 40 50
60 100 120 140 180
Output: 280

Answer

```
from queue import PriorityQueue
```

```

def bound(u, n, W, items):
    if u[1] >= W: return 0
    result, wt, i = u[2], u[1], u[0]+1
    while i < n and wt + items[i][0] <= W:
        wt += items[i][0]
        result += items[i][1]
        i += 1
    if i < n: result += (W - wt) * (items[i][1]/items[i][0])
    return result

def knapsack(n, W, weights, values):
    items = sorted(zip(weights, values), key=lambda x: x[1]/x[0], reverse=True)
    Q = PriorityQueue()
    Q.put((-bound((-1,0,0), n, W, items), (-1,0,0))) # (level, weight, value)
    max_val = 0
    while not Q.empty():
        _, u = Q.get()
        level, wt, val = u
        if level == n-1: continue
        # Include next item
        next_wt, next_val = wt + items[level+1][0], val + items[level+1][1]
        if next_wt <= W: max_val = max(max_val, next_val)
        if bound((level+1, next_wt, next_val), n, W, items) > max_val:
            Q.put((-bound((level+1, next_wt, next_val), n, W, items), (level+1, next_wt,
next_val)))
        # Exclude next item
        if bound((level+1, wt, val), n, W, items) > max_val:
            Q.put((-bound((level+1, wt, val), n, W, items), (level+1, wt, val)))
    return max_val

# Input
n, W = map(int, input().split())
weights = list(map(int, input().split()))
values = list(map(int, input().split()))

print(knapsack(n, W, weights, values))

```

Status : Correct

Marks : 10/10

4. Problem Statement

Josh is learning about the Fractional Knapsack problem, where a knapsack has a fixed maximum capacity. Given a set of items, each with a value and a weight, Josh wants to maximize the total value placed into the knapsack.

Unlike the 0/1 Knapsack problem, fractions of items are allowed. Items must be selected using a Greedy strategy based on their value-to-weight ratio.

Write a program to determine the fraction of each item that should be included in the knapsack to achieve the maximum total value.

Formula

Value-to-Weight Ratio

$\text{ratio} = \text{value} / \text{weight}$

Fraction Calculation

$\text{fraction} = \text{remainingCapacity} / \text{weight}$

Notes

Items must be selected using the Greedy approach based on value-to-weight ratio. Sorting is required to determine the optimal order. Output fractions must be printed in the original item order. Use double datatype only for calculations.

Input Format

The first line contains an integer n , representing the number of items.

The second line contains n space-separated integers representing item values.

The third line contains n space-separated integers representing item weights.

The fourth line contains an integer capacity, representing knapsack capacity.

Output Format

Print n space-separated double values, representing the fraction of each item selected.

Output must be printed rounded to exactly two decimal places.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

3 5 1 2 4

40 50 20 10 30

75

Output: 0.00 0.70 0.00 1.00 1.00

Answer

```
# You are using Python
```

```
# Read input
```

```
n = int(input())
```

```
values = list(map(int, input().split()))
```

```
weights = list(map(int, input().split()))
```

```
capacity = int(input())
```

```
# Calculate value-to-weight ratio and keep original index
```

```
items = [(values[i]/weights[i], weights[i], i) for i in range(n)]
```

```
# Sort items by ratio in descending order
```

```
items.sort(key=lambda x: x[0], reverse=True)
```

```
fractions = [0.0] * n
```

```
remaining = capacity
```

```
# Greedy selection
```

```
for ratio, weight, idx in items:
```

```
    if remaining >= weight:
```

```
        fractions[idx] = 1.0
```

```
        remaining -= weight
```

```
    else:
```

```
        fractions[idx] = remaining / weight
```

```
        remaining = 0
```

```
    break
```

```
# Print fractions rounded to 2 decimal places
```

```
print(" ".join(f"{f:.2f}" for f in fractions))
```


Status : Correct

Marks : 10/10

5. Problem Statement

Ananya is preparing for a long trek and has a bag with limited capacity. She has several items, each with a weight and value, and she wants to maximize the total value she can carry. She may take whole items or fractions of them to fit in her bag.

The maximum value is calculated using the formula:

$$\text{Maximum Value} = \sum (\text{value}[i] * \text{fraction_taken}[i])$$

where $\text{fraction_taken}[i] = \min(1, \text{remaining_capacity} / \text{weight}[i])$

Write a program to help Ananya determine the maximum achievable value.

Input Format

The input consists of several lines, each representing an item.

Each line (until -1) contains two integers: weight and value of an item.

The input terminates with -1.

The final line contains one integer representing the bag's capacity.

Output Format

The output prints "The maximum value of the current list is:"

On the next line, print the maximum value achievable with exactly two decimal places

Refer to the sample outputs for the formatting specifications.

Sample Test Case

Input: 10 60
20 100

30 120

-1

50

Output: The maximum value of the current list is:

240.00

Answer

```
# You are using Python
```

```
# Read items
```

```
items = []
```

```
while True:
```

```
    line = input().strip()
```

```
    if line == "-1":
```

```
        break
```

```
    w, v = map(int, line.split())
```

```
    items.append((w, v))
```

```
# Read capacity
```

```
capacity = int(input())
```

```
# Compute value-to-weight ratio and sort descending
```

```
items.sort(key=lambda x: x[1]/x[0], reverse=True)
```

```
max_value = 0.0
```

```
remaining = capacity
```

```
for weight, value in items:
```

```
    if remaining >= weight:
```

```
        max_value += value
```

```
        remaining -= weight
```

```
    else:
```

```
        max_value += value * (remaining / weight)
```

```
        break
```

```
# Print result
```

```
print("The maximum value of the current list is:")
```

```
print(f"{max_value:.2f}")
```

Status : Correct

Marks : 10/10